

Tab 1

1 Digital Forensics – Definition

Digital Forensics is the scientific process of identifying, acquiring, preserving, analyzing, and presenting digital evidence in a legally admissible manner.

It ensures:

- Integrity of digital evidence
- Proper documentation
- Court admissibility

 Focus: Maintaining **chain of custody** and preventing evidence tampering.

2 Digital Forensics Process

The process follows a structured methodology:

1. Identification

- Detect digital devices involved (laptops, mobiles, servers, IoT devices, USB drives).
- Identify potential sources of evidence.

2. Preservation

- Isolate devices from network.
- Create forensic image (bit-by-bit copy).
- Use **write blockers** to avoid modification.

3. Collection / Acquisition

- Acquire volatile data (RAM, network sessions).
- Acquire non-volatile data (hard disk, SSD).

4. Examination

- Recover deleted files.
- Extract logs, metadata, timestamps.
- Keyword searches.

5. Analysis

- Correlate evidence.
- Timeline reconstruction.
- Identify attack vectors.

6. Presentation

- Prepare forensic report.
 - Expert testimony in court.
 - Evidence explanation in simple language.
-

3 Locard's Principle of Exchange

Proposed by **Edmond Locard**.

Principle:

“Every contact leaves a trace.”

In Digital Context:

Whenever a cybercrime occurs:

- Attacker leaves digital footprints (IP logs, metadata, artifacts).
- Victim system records traces (event logs, registry entries).
- Malware leaves signatures or persistence mechanisms.

Example:

- USB insertion leaves registry trace.
- Remote login leaves authentication logs.

Even deleting data leaves remnants in unallocated space.

 Key Exam Line:

Digital forensics is built on the foundation of Locard's principle.

4 Branches of Digital Forensics

♦ **1. Computer Forensics**

- Hard disks, operating systems
- File system analysis

♦ **2. Network Forensics**

- Packet capture analysis
- Firewall logs
- IDS/IPS alerts

♦ **3. Mobile Forensics**

- Call logs
- App data
- GPS location

♦ **4. Memory (RAM) Forensics**

- Volatile data analysis
- Running processes
- Encryption keys

♦ **5. Database Forensics**

- SQL logs
- Data manipulation tracking

♦ **6. Cloud Forensics**

- Virtual machines
- SaaS logs
- API activity

♦ **7. IoT Forensics**

- Smart devices
 - Embedded systems
-

5 Handling Digital Crime Scene

Unlike physical crime scenes, digital scenes require extra caution.

Step-wise Handling:

1. Secure the Scene

- Prevent unauthorized access.
- Disconnect from network (if required).

2. Document Everything

- Photograph setup.
- Record device status (ON/OFF).
- Note connected peripherals.

3. If Device is ON:

- Capture volatile memory first.
- Do NOT shut down immediately.

4. If Device is OFF:

- Do NOT power it ON.
- Seize device properly.

5. Use Proper Packaging

- Anti-static bags.
- Faraday bags (for mobiles).

6. Maintain Chain of Custody

- Record who handled evidence.
- Time and date of transfer.



Mistake to Avoid:

- Never explore files casually.
 - Never plug suspect USB into personal laptop.
-

6 Important Documents in Digital Forensics

1. Chain of Custody Form

- Tracks evidence movement.
- Ensures integrity.

2. Seizure Memo

- Lists confiscated items.
- Signed by witnesses.

3. Forensic Imaging Report

- Hash values (MD5, SHA-256).
- Imaging tool used.

4. Analysis Report

- Findings.
- Artifacts recovered.
- Timeline events.

5. Expert Opinion Report

- Technical explanation for court.
-

7 Electronic Evidence

Definition:

Any information stored or transmitted in digital form that can be used in court.

Types:

- Emails
- Chat logs

- Browser history
- CCTV footage
- Server logs
- Metadata
- Deleted files
- GPS records
- Social media data

Characteristics:

- Fragile
- Easily altered
- Requires forensic acquisition

Legal Admissibility Depends On:

- Proper collection
- Hash verification
- Chain of custody
- Documentation



Quick Revision Table

Topic	Key Focus
Digital Forensics	Scientific handling of digital evidence
Locard's Principle	Every action leaves digital trace
Process	Identification → Preservation → Collection → Analysis → Presentation
Crime Scene	Do not alter evidence
Documents	Chain of custody is critical
Electronic Evidence	Must be authentic and verified

Perfect. This is a **core digital forensics unit** — exam favorite + practical heavy.
 Let's structure it cleanly for a **15–20 marker** answer.

1 Introduction to Evidence Acquisition

Evidence acquisition is the **systematic process of collecting digital evidence** without altering its original state so that it remains legally admissible.

Digital evidence is fragile — even opening a file changes metadata. That's why procedure matters more than tools.

◆ A. Identification

First step: determine **what can contain evidence**.

Identify:

- Computers (desktops, laptops)
- Mobile devices
- External drives (USB, HDD, SSD)
- Cloud accounts
- Network devices (routers, firewalls)
- IoT devices

Also Identify:

- Volatile data (RAM, running processes)
- Non-volatile data (disk storage)

📌 Priority Rule:

Volatile evidence must be captured first (RAM → network sessions → disk).

◆ B. Acquisition

Acquisition means **creating a forensic copy (bit-by-bit image)** of digital media.

Types of Acquisition:

1. **Live Acquisition**
 - System is ON
 - Used for volatile data (RAM, encryption keys)
 - Risk: May alter system
 2. **Dead Acquisition**
 - System is OFF
 - Disk removed and imaged
 - Safer and preferred method
 3. **Logical Acquisition**
 - Selected files only
 4. **Physical Acquisition**
 - Full bit-stream copy (recommended)
-

◆ C. Labeling and Packaging

After seizure:

Label Includes:

- Case number
- Description of device
- Date & time seized
- Investigator name
- Location

Packaging:

- Anti-static bags (for HDD/SSD)
 - Faraday bags (for mobile devices)
 - Tamper-evident seals
-

◆ D. Transportation

Evidence must be:

- Protected from physical damage
- Shielded from electromagnetic interference
- Properly logged during transfer

Never:

- Leave evidence unattended
 - Transport loosely packed devices
-

◆ E. Chain of Custody

Chain of custody is the **chronological documentation of evidence handling**.

It records:

- Who collected it
- When it was collected
- Who accessed it
- Purpose of access
- Time of transfer

📌 Why Important?

- Prevents tampering allegations
 - Ensures court admissibility
 - Maintains integrity
-

◆ F. Importance of Documentation & Preservation

Documentation ensures:

- Transparency
- Reproducibility
- Legal compliance

Preservation prevents:

- Data corruption
- Overwriting
- Metadata alteration

Every action must be logged.

2 Acquisition Process – Technical Aspects

Now the deeper technical layer.

◆ A. Write-Blockers

A **write-blocker** prevents modification of original media during acquisition.

Types:

- Hardware write-blocker (preferred)
- Software write-blocker

Why Required?

Without it:

- System may modify timestamps
 - OS may create hidden files
 - Evidence becomes legally questionable
-

◆ B. Imaging Techniques

Imaging = Creating an exact forensic duplicate.

1. Bit-Stream Imaging

- Full sector-by-sector copy
- Includes deleted & slack space
- Most reliable

2. Disk-to-Image File

- Creates image file (.E01, .dd)

3. Disk-to-Disk Clone

- Copy from suspect disk to another disk

◆ C. Evidence Integrity

Integrity ensures evidence is unchanged.

How Maintained?

- Hashing (MD5, SHA-1, SHA-256)
- Compare hash before and after imaging

If:

Original Hash = Acquired Image Hash

➡ Evidence integrity preserved

◆ D. Standard Operating Procedures (SOPs)

SOPs define:

1. Secure crime scene
2. Identify devices
3. Document environment
4. Capture volatile data
5. Remove storage device
6. Attach write-blocker
7. Create forensic image
8. Generate hash
9. Seal and store original media
10. Work only on forensic copy

📌 Golden Rule:

Always analyze the copy, never the original.

3 Introduction to Data Recovery & Carving

Now we move to post-acquisition analysis.

◆ A. Importance of Data Recovery in Forensic Investigation

Data recovery helps retrieve:

- Deleted files
- Formatted data
- Corrupted files
- Hidden partitions

Why critical?

- Criminals attempt deletion
 - Formatting does not erase data immediately
 - Key evidence may exist in unallocated space
-

◆ B. Data Recovery

Data recovery uses:

- File system metadata
- Backup tables (FAT, MFT)
- Journals

It works when:

- File structure still exists
- Metadata intact

Example:

Recovering deleted file from Recycle Bin or MFT entry.

◆ C. Data Carving

Data carving retrieves files **without relying on file system metadata**.

It works by:

- Searching file headers & footers
- Identifying file signatures (magic numbers)

Example:

- JPEG starts with FF D8 FF
- PDF starts with %PDF

Even if file system is destroyed, carving can recover file fragments.

Difference Between Data Recovery & Data Carving

Feature	Data Recovery	Data Carving
Uses File System Metadata	Yes	No
Works After Formatting	Sometimes	Yes
Speed	Faster	Slower
Accuracy	Higher if metadata intact	May recover fragmented files
Use Case	Normal deletion	Severe corruption or wiping

[Understanding the FAT file system](#)
[File Allocation Table\(FAT\)](#)
[FAT Filesystem](#)

Tab 2

File System Overview (Digital Forensics)

1. What is a File System?

A **file system** is a method used by an operating system to **store, organize, manage, and retrieve data from storage devices** such as hard disks, SSDs, or USB drives.

A file system stores data on a device so it can be retrieved by the system or user.

Key Idea

Think of a file system like a **library management system**.

Library	File System
Books	Files
Shelf location	Disk block location
Catalog index	Metadata
Librarian	Operating System

Without a file system, the OS **cannot locate or manage files on disk**.

2. Relationship Between File System and Operating System

A file system is **mostly independent of the operating system**.

Different operating systems can support multiple file systems **if proper drivers are installed**.

Examples

Operating System	Supported File Systems
Windows	NTFS, FAT32, exFAT

Linux	EXT2, EXT3, EXT4
macOS	APFS, HFS+

Cross Support Example

- Linux can read/write **NTFS**
- Windows can read Linux **EXT** with drivers.

File systems are **OS independent logical structures used to manage data storage on physical media**.

3. File System Formatting and Volumes

File systems are created when a storage device is **formatted**. File systems are formatted onto a **volume**.

What is a Volume? A **volume** is a logical storage unit.

It can be:

① Single Partition Volume

Example: Hard Disk → Partition C → NTFS File System

② Multiple Partition Volume

Used in:

- RAID
- LVM (Logical Volume Manager)

Example: Multiple disks → Combined → Single logical volume

4. Purpose of File Systems

A file system allows the operating system to **access files stored on disk**.

1. Store data on storage media
2. Organize files and directories
3. Track file locations
4. Manage metadata

5. Control access and permissions
6. Enable retrieval of files quickly
7. Maintain file integrity

5. File System as a Database

Because file systems store:

Type	Meaning
Data	Actual file content
Metadata	Information about the file

Example: File: report.pdf

Data: Actual PDF contents

Metadata:

File name

Size

Creation time

Location on disk

Owner

Permissions

File systems store **data and metadata**, similar to how databases store records and attributes.

6. Metadata in File Systems

Metadata = Data about data

Metadata includes:

- File location
- File size
- File name
- Path
- permissions

- timestamps

Examples of Metadata

Metadata Field	Meaning
File name	Name of file
Size	File size
Location	Disk address
Created time	Creation timestamp
Modified time	Last modification
Permissions	Access rights

Digital Forensics Importance

Metadata helps investigators:

- Determine **when a file was created**
- Identify **file modifications**
- Track **user activity**

7. File Data Storage – Blocks and Clusters

File data is stored in **blocks or clusters**. File systems track file locations using **block addresses**.

Basic Storage Structure

Disk
↓
Sector
↓
Cluster / Block
↓
File

8. Sector, Block, and Cluster

1 Sector

Smallest unit on disk. Typical size: 512 bytes or 4096 bytes

2 Cluster

Group of sectors. Used mainly in **Windows file systems**.

3 Block

Equivalent concept used in **Linux file systems**.

Comparison

Term	Used In	Description
Sector	Hardware	Smallest storage unit
Cluster	Windows	Group of sectors
Block	Linux	Group of sectors

Microsoft file systems use clusters, while Linux refers to them as blocks.

9. Cluster Size Example (Important)

Typical NTFS cluster size: 4096 bytes

Which equals: 8 sectors

Because: 1 sector = 512 bytes

Therefore: $8 \times 512 = 4096$ bytes

10. Cluster Size Impact

If a file is **1 byte**, it still occupies: 4096 bytes

Why? Because the **minimum allocation unit is one cluster**.

Example

File Size	Disk Space Used
1 byte	4096 bytes
2000 bytes	4096 bytes
5000 bytes	8192 bytes

This unused space is called:

Slack Space

Very important in **digital forensics**.

11. Why Clusters are Used

The slides explain that clusters help **manage large drives efficiently**.

Advantages:

1. Reduces number of addresses
2. Improves disk performance
3. Simplifies storage management
4. Allows larger disks to be supported

12. Types of File System Data

File system data can be divided into two categories.

1 Essential File System Data

Essential data is required to **store and retrieve files**.

- File content addresses
- File name
- Pointer to metadata structure

Without essential data

The OS **cannot access files**.

2 Non-Essential File System Data

This information is useful but **not necessary for basic operation**.

Examples:

- Access timestamps
- File permissions

If lost

Files can still be accessed, but **some attributes disappear**.

13. Types of File System Information

1 General File System Information

Includes:

- Structure of file system
- Location of data structures
- Size of storage units

Example:

Superblock

Boot sector

Volume information

2 Content of Files

Actual file data stored inside containers such as:

- clusters
- blocks

Example:

Images
Documents
Programs
Videos

3 File Metadata Structures

Examples listed in the slides:

File System	Metadata Structure
FAT	Directory entry
NTFS	MFT entry
EXT	Inode

These structures store file attributes.

4 File Names

Metadata includes file names and their location in directory hierarchy.

Example path: C:\Users\Aastha\Documents\report.pdf

5 User Quota Statistics

Tracks **how much disk space each user consumes**.

Example:

User A → 5GB used
User B → 2GB used

6 File System Journals

Used for **recovery and integrity**.

Example:

- NTFS journal
- EXT3 journal

Purpose:

- Recover from system crashes
- Maintain file consistency

14. File Name Limitations

Older file systems had restrictions.

Example from slide:

FAT16

Filename format: 8.3 format

Meaning: 8 characters + 3 extension

Example: REPORT.TXT

Modern file systems support:

Long filenames

Example: CyberSecurityInvestigationReport.pdf

But there are still **path length limits** depending on OS.

15. Common File Systems

The slide lists common file systems used in different operating systems.

1 NTFS

Used in: Windows

Features:

- journaling
- security permissions
- large file support

- MFT metadata

2 FAT File System

Older Windows file system.

Examples:

FAT12

FAT16

FAT32

Used in:

- USB drives
- memory cards

3 Unix File Systems

Examples:

EXT

EXT2

EXT3

EXT4

Used in: Linux

Metadata structure: inode

4 Apple File Systems

Examples:

HFS+

APFS

Used in: macOS

5 Other File Systems

Examples:

UFS

CDFS

Used in:

- Unix
- CD file systems

16. Memory Trick for Exam

Remember file system components using:

DMBFCJ

D → Data
M → Metadata
B → Blocks
F → File names
C → Clusters
J → Journals

17. Perfect 10-Marker Answer Structure

If asked: “**Explain File System Overview**”

Write in this order:

- 1 Definition of file system
- 2 Relationship with OS
- 3 File system formatting and volumes
- 4 Data vs Metadata
- 5 Blocks, sectors, clusters
- 6 Cluster size and slack space
- 7 Essential vs Non-essential data
- 8 File metadata structures
- 9 File naming limitations
- 10 Types of file systems

Digital Forensics – Part 3

This part mainly covers three major concepts:

1. **How Data is Stored on Disk**
2. **Slack Space**
3. **Data Carving**
4. **Data Acquisition**

These topics are **very important in digital forensic investigations**.

1. How Data is Stored on a Disk

According to the slide (page 3), traditional **magnetic hard disks store data using cylinders, heads, and sectors**.

Disk Structure

Hard disks contain **circular magnetic platters** where data is stored.

Disk Organization

Hard Disk



Platter



Track



Sector



Cluster

1.1 Platter

A **platter** is a circular magnetic disk inside the hard drive.

Characteristics:

- Stores actual data

- Multiple platters exist in a disk
- Each platter has **two surfaces**

Example

Hard Disk

|— Platter 1

|— Platter 2

|— Platter 3

1.2 Tracks

Tracks are **concentric circular paths** on the platter where data is written.

From the slide:

Concentric circles of storage areas on each side of the platter are called tracks.

Example

Platter

| Track 1

| Track 2

| Track 3

| Track 4

Each track stores data.

1.3 Sectors

Tracks are divided into **sectors**.

A **sector is the smallest physical storage unit** on a disk.

Typical sector size:

512 bytes

or

4096 bytes

According to the slide: Tracks were limited to 63 sectors historically.

Example:

Track

└ Sector 1

└ Sector 2

└ Sector 3

└ Sector 63

1.4 Cylinder

A **cylinder** is a group of tracks located at the same radius across all platters.

Example: Cylinder = Track 1 on all platters

Visualization:

Platter 1 → Track 1

Platter 2 → Track 1

Platter 3 → Track 1

Together they form **Cylinder 1**.

2. Cluster Storage (Important Concept)

Even though disks store data in **sectors**, the **operating system stores files in clusters**.

What is a Cluster?

A **cluster** is a group of sectors.

Example

1 Cluster = 8 Sectors

If: 1 sector = 512 bytes

Then: 1 cluster = 4096 bytes (4 KB)

Clusters help manage storage more efficiently.

3. Slack Space

Slack space is the difference between the allocated space and the space actually used by the file.

Example

Cluster size: 4096 bytes

File size: 1000 bytes

Disk usage: 4096 bytes

Unused space: $4096 - 1000 = 3096$ bytes

This unused space is called **Slack Space**.

Types of Slack Space

1. RAM Slack

Space between: End of file → End of sector

2. File Slack

Space between: End of sector → End of cluster

Why Slack Space Matters in Digital Forensics

Slack space may contain:

- fragments of deleted files
- previous data remnants
- hidden malicious content

Investigators often analyze slack space to find **hidden evidence**.

4. Slack Space Example (From Slide)

The slide provides a **one-byte file example**.

If a **1 byte file is created**:

The NTFS system writes: 1 byte actual data + 511 null bytes
to fill the sector.

If cluster size is **4 KB (8 sectors)**:

Only the first sector is overwritten.

Remaining sectors may still contain **old deleted data**.

This leftover data becomes **cluster slack**.

5. Data Carving

Data carving is one of the **most powerful forensic techniques**.

Definition from the slide: Data carving is the act of searching for particular strings or bytes within a structure.

Simple Explanation

Data carving means: Recovering files by searching for known data patterns.

Even if:

- file system is damaged
- file table is deleted
- file is hidden

Investigators can still recover data.

Tools Used for Data Carving

Examples:

- Hex Editor
- Autopsy
- Foremost
- Scalpel
- Photorec

These tools search raw disk data.

6. How Data Carving Works

Steps:

1. Investigator identifies a **file signature**
2. Searches disk for that signature
3. Extracts file between header and footer

Example:

JPEG file signature

Header → FF D8

Footer → FF D9

If found: FF D8 ... FF D9

The tool extracts the image.

7. Scope of Data Carving

According to the slide, carving can be done in multiple places.

Possible targets:

- Individual file
- Slack space
- Unallocated disk space
- Full disk volume
- Memory image
- Swap file

This flexibility makes carving **very powerful**.

8. File Carving

The slide explains that **files are carved using file headers/signatures**.

Example

File Type	Header
JPEG	FF D8
PNG	89 50 4E 47
PDF	25 50 44 46

The tool finds the header and reconstructs the file.

Limitation of File Carving

File carving works best when files are: Contiguous

Meaning: stored in one continuous block

If a file is **fragmented**, carving becomes difficult.

Advanced Technique

The slide mentions **Fuzzy Hashing** for fragmented files.

Fuzzy hashing compares **similar file fragments** to reconstruct files.

Example tools:

- ssdeep
- sdhash

9. Record Carving

Record carving means recovering **specific pieces of information**, not full files.

Example from slide:

Recovering a **web log entry**.

A single log entry may contain:

- date
- time
- IP address
- webpage visited
- browser ID

Even if the file is deleted, the **record might still exist in disk sectors**.

10. Example Case from Slide

A real investigation example is given in the slide.

Scenario: Law enforcement seized a system.

Problems:

- System logs were deleted
- Very little logical evidence remained

Investigator used: Data carving

Result: Recovered **22,000 Windows Event Log entries**.

Recovered information included:

- IP addresses
- Hostname
- Date and time of infection

These logs helped identify **malware activity**.

11. Data Acquisition

Data acquisition is the **first step in digital forensic investigation**.

Definition: Data acquisition is the process of collecting data relevant to an investigation.

Types of Data Acquisition

1. Live Acquisition

Data collected from **running systems**.

Examples:

- RAM capture
- network connections
- running processes

Used when the system **cannot be powered off**.

2. Dead Acquisition

Data collected from **powered-off systems**.

Example: Disk imaging

This method is more reliable.

12. Forensically Sound Acquisition

The slide emphasizes: Acquisition should create the fewest changes to the system.

This ensures evidence integrity.

Principles:

- Do not modify original data
- Maintain evidence authenticity
- Document every step

13. Write Blocker

During disk acquisition, a **write blocker** is used.

Purpose: Prevent modification of original disk.

Process from slide:

1. Remove hard drive from system
2. Connect it to **write blocker**
3. Connect write blocker to forensic workstation
4. Create forensic image

14. Disk Imaging Process

Steps:

Suspect Drive



Write Blocker



Forensic Workstation



Target Storage Device

A **forensic image** is created.

Example formats:

- E01
- DD
- RAW

15. Chain of Custody

After acquisition: Original evidence must be documented.

The slide mentions maintaining **chain of custody**.

Chain of custody records:

- who collected evidence
- when it was collected
- where it was stored
- who accessed it

This ensures **legal admissibility in court**.

16. Examination of Forensic Image

Investigators **do not analyze the original disk**.

Instead they analyze: Forensic image

Advantages:

- prevents evidence alteration
- allows repeated analysis
- preserves original evidence

17. Read-Only Analysis

The slide highlights that analysis is done in **read-only mode**.

Benefits:

- evidence integrity maintained
- experiments repeatable
- no accidental modification

Digital Forensics

1. What is Digital Forensics?

Digital forensics is a branch of forensic science that focuses on **identifying, collecting, analyzing, and presenting digital evidence** in a manner that is legally acceptable in court.

In simple terms, it is the **scientific investigation of digital devices** to determine how a cybercrime occurred and who was responsible.

Digital forensics combines:

- computer science

- investigation techniques
- legal procedures
- mathematical validation methods

The goal is to analyze digital evidence in a way that ensures:

- authenticity
- integrity
- reliability
- admissibility in court

Digital evidence may come from many sources such as:

- computers
- mobile phones
- servers
- cloud platforms
- networks
- storage devices

Digital forensics ensures that evidence is **collected, preserved, and analyzed without altering the original data.**

2. Definition of Digital Forensics

Digital forensics can be defined as:

The process of preservation, identification, extraction, and documentation of digital evidence so that it can be used in a court of law.

This definition highlights four key stages:

1. **Preservation** – Protecting evidence from modification
2. **Identification** – Locating potential digital evidence
3. **Extraction** – Collecting data from digital sources
4. **Documentation** – Recording investigation results

Digital forensics is therefore both a **technical and legal discipline.**

3. Scope of Digital Forensics

Digital forensics covers a wide range of digital systems and technologies.

It is not limited to desktop computers. The scope includes:

1. Computers

Analysis of:

- hard drives
- file systems
- deleted files
- system logs

2. Mobile Devices

Investigation of:

- call logs
- SMS messages
- application data
- location information

3. Network Systems

Analysis of:

- network traffic
- intrusion attempts
- packet captures

4. Cloud Systems

Investigation of:

- cloud storage
- user activity
- access logs

5. Multimedia Data

Analysis of:

- images
- videos
- audio recordings

The objective of multimedia analysis is often:

- authentication
- comparison
- enhancement

For example, investigators may analyze a video to determine if it has been manipulated.

4. Objectives of Computer Forensics

The primary objectives of digital forensics focus on **investigation and evidence handling**.

1. Identifying the Criminal

Digital forensic analysis helps determine:

- who committed the crime
- how the crime occurred
- when the attack happened
- what tools were used

This helps investigators understand the **motive and identity of the culprit**.

2. Protecting Digital Evidence

Digital forensics ensures that evidence is collected carefully so that it is **not corrupted or altered**.

Investigators design procedures to:

- secure digital devices
- prevent tampering
- preserve data integrity

3. Recovery of Deleted Data

Forensic techniques allow investigators to recover:

- deleted files
- deleted partitions
- hidden data

This helps reveal evidence that suspects attempted to destroy.

4. Evidence Preservation

Digital evidence must be preserved so it can be used in court.

This includes:

- maintaining chain of custody
- storing evidence securely
- documenting all procedures

5. Impact Analysis

Forensic investigation helps determine:

- how much damage was caused
- which systems were affected
- how attackers accessed the system

This helps organizations understand the **impact of malicious activity**.

6. Investigation Reporting

A complete forensic investigation produces a **forensic report**.

The report includes:

- investigation methodology
- tools used
- findings
- conclusions

This report is used as **legal evidence**.

5. Types of Digital Forensics

Digital forensics is divided into several specialized branches depending on the type of digital evidence.

1. Disk Forensics

Disk forensics focuses on analyzing **storage media**.

Examples:

- hard disks
- SSDs
- USB drives
- memory cards

Investigators analyze:

- active files
- modified files
- deleted files

The goal is to recover evidence from storage devices.

2. Network Forensics

Network forensics involves monitoring and analyzing **network traffic**.

Investigators study:

- packet captures
- network logs
- intrusion attempts

This helps detect:

- hacking attacks
- data theft
- unauthorized access.

3. Wireless Forensics

Wireless forensics focuses on **wireless network communications**.

Examples include:

- WiFi traffic
- Bluetooth communications
- wireless access points

Investigators analyze wireless traffic to detect suspicious activity.

4. Database Forensics

Database forensics involves analyzing **database systems and metadata**.

Investigators examine:

- database logs
- user queries
- transaction records

This helps identify unauthorized database access or manipulation.

5. Malware Forensics

Malware forensics involves analyzing **malicious software**.

Examples of malware:

- viruses
- worms
- trojans
- ransomware

Investigators analyze malware to understand:

- how it spreads
- what damage it causes
- what payload it executes.

6. Email Forensics

Email forensics focuses on analyzing email communications.

Investigators examine:

- email headers
- attachments
- deleted emails
- contact lists

This helps track:

- phishing attacks
- email fraud
- communication between criminals.

7. Memory Forensics

Memory forensics involves analyzing **RAM (Random Access Memory)**.

RAM may contain:

- running processes
- encryption keys
- malware artifacts
- network connections

Investigators capture a **memory dump** and analyze it to extract valuable information.

8. Mobile Forensics

Mobile forensics focuses on smartphones and tablets.

Investigators recover:

- call logs
- SMS messages
- contacts
- photos
- videos
- application data

Mobile forensics is increasingly important because smartphones store **large amounts of personal information**.

6. Advantages of Digital Forensics

Digital forensics provides several benefits in cybercrime investigation.

1. Maintains System Integrity

It helps ensure that computer systems remain secure and evidence remains intact.

2. Legal Evidence

Digital forensics produces evidence that can be presented in court to prosecute criminals.

3. Detecting Cybercriminals

Investigators can track cybercriminals even if they operate from different countries.

4. Protecting Organizations

Organizations use digital forensics to investigate cyberattacks and protect their systems.

5. Time and Cost Savings

Quick identification of attacks reduces financial damage and operational downtime.

6. Accurate Evidence Analysis

Digital forensics helps extract and interpret factual evidence from digital devices.

This evidence helps prove cybercriminal activities in court.

7. Disadvantages of Digital Forensics

Despite its benefits, digital forensics also has several challenges.

1. Evidence Authenticity Requirement

Digital evidence must be proven to be **untampered and authentic** before it is accepted in court.

2. High Cost

Digital forensic investigations require:

- specialized software
- expensive hardware
- skilled professionals

This makes investigations costly.

3. Requirement of Technical Expertise

Investigators must possess strong knowledge of:

- computer systems
- networking
- operating systems
- programming

Without technical expertise, evidence may be misinterpreted.

4. Tool Reliability

If the forensic tool used is not validated or certified, the evidence may be rejected in court.

5. Lack of Investigator Knowledge

If investigators lack technical knowledge, important evidence may be overlooked.

8. Digital Forensic Investigation Process

Digital forensic investigation follows a structured methodology.

The process includes five major stages.

1. Identification

The first step is identifying potential digital evidence.

This includes:

- identifying devices involved in the crime
- determining investigation objectives
- identifying required resources

Examples of evidence sources:

- computers
- mobile devices
- servers
- network logs

2. Preservation

In this stage, investigators secure the digital evidence to prevent modification.

Activities include:

- isolating devices
- creating forensic images
- protecting evidence from tampering

Evidence must be preserved to maintain its **integrity and authenticity**.

3. Analysis

The collected data is analyzed using forensic tools.

Tasks include:

- recovering deleted files
- analyzing logs
- examining system artifacts
- identifying suspicious activity

This step helps investigators reconstruct the sequence of events.

4. Documentation

Every step of the investigation must be documented.

Documentation includes:

- evidence collection procedures
- tools used
- analysis results
- screenshots and logs

Proper documentation ensures transparency and reproducibility.

5. Presentation

In the final stage, investigators present their findings.

This may involve:

- writing forensic reports
- explaining evidence
- presenting findings in court

The objective is to clearly explain the evidence and conclusions.

Windows Registry (Digital Forensics)

1. What is the Windows Registry?

The **Windows Registry** is a **central hierarchical database** in Microsoft Windows operating systems that stores configuration settings and options for the operating system, hardware devices, installed software, and user preferences.

In simple terms, the registry is like the **brain or configuration center of Windows**.

It stores information about:

- operating system configuration
- installed applications
- hardware devices
- user accounts
- system security settings
- user activity

The registry allows Windows to quickly retrieve configuration data when required.

2. Purpose of the Windows Registry

The registry serves several important purposes in a Windows system:

1. System Configuration Storage

It stores configuration settings related to:

- system hardware

- operating system settings
- device drivers
- network configuration.

2. Application Configuration

Applications store settings such as:

- installation paths
- preferences
- license information.

3. User Profile Data

User-specific data is also stored, including:

- desktop settings
- browser history artifacts
- recently accessed files.

4. Activity Tracking

Registry entries record user activities such as:

- executed programs
- accessed files
- connected devices.

Because of this, the registry is **extremely valuable in forensic investigations**.

3. Registry Structure

The registry follows a **tree-like hierarchical structure** similar to a file system.

Structure hierarchy:

Registry

↓

Hive

↓

Key



Subkey



Value



Data

The main components of the registry are:

- Hives
- Keys
- Subkeys
- Values
- Data

4. Key Terminology in the Registry

1. Registry

The registry is a **collection of database files that store system and user configuration information**.

2. Registry Editor

Registry Editor is a Windows utility used to **view and modify registry entries**.

Two tools are commonly used:

- **Regedit**
- **Regedt32**

These tools allow investigators or administrators to analyze registry contents.

3. HKEY

The registry is divided into categories called **HKEY**, which act as the top-level containers.

Example:

HKEY_LOCAL_MACHINE

HKEY_CURRENT_USER

The letter **H** represents a **handle to a key**.

4. Key

A **key** is similar to a folder.

It can contain:

- subkeys
- values.

5. Subkey

A **subkey** is a key located inside another key.

It functions like a **subfolder in a directory structure**.

6. Branch

A **branch** refers to a key and all of its subkeys.

Example:

HKEY_LOCAL_MACHINE

└ Software

└ Microsoft

This entire path forms a branch.

7. Value

A **value** contains the actual stored data.

Each value includes:

- name
- data
- data type.

Values are similar to **files inside folders**.

8. Default Value

Every registry key contains a **default value** which may or may not contain data.

5. Registry Hives

A **registry hive** is a major section of the registry database.

Hives are stored as files on disk and loaded into memory when the system starts.

The main registry hives are:

1. HKEY_CLASSES_ROOT
2. HKEY_CURRENT_USER
3. HKEY_LOCAL_MACHINE
4. HKEY_USERS
5. HKEY_CURRENT_CONFIG

Registry hive files are stored in **4 KB blocks called bins** and typically begin with the header **regf**.

6. Important Registry Files

Registry information is stored in several files.

System Registry Files

Location:

Windows\System32\Config

Important files include:

- SYSTEM

- SOFTWARE
- SAM
- SECURITY
- DEFAULT

User-specific files include:

- NTUSER.DAT
- USRCLASS.DAT

These files are crucial for forensic investigations.

7. Important Registry Files Explained

1. SOFTWARE

Location:

Windows\System32\Config\SOFTWARE

Contains:

- installed software information
- application configuration data
- Windows operating system configuration.

Applications store their settings inside this hive.

On 64-bit systems, compatibility entries are stored under:

WOW6432Node

2. SYSTEM

Location:

Windows\System32\Config\SYSTEM

Contains system configuration such as:

- device drivers
- control sets
- hardware configuration.

It includes multiple **control sets**, such as:

- Current Control Set
- Last Known Good Configuration.

These help recover the system if configuration becomes corrupted.

3. SAM (Security Accounts Manager)

Location:

Windows\System32\Config\SAM

Contains information about:

- local user accounts
- encrypted passwords
- user identifiers.

This hive is extremely important during forensic analysis because it contains authentication information.

Domain account information is stored in **Active Directory**, not locally.

4. SECURITY

Contains security policy information such as:

- access control policies
- domain security settings

- user privileges.

The Windows kernel reads this information to enforce security rules.

5. NTUSER.DAT

Location:

C:\Users\Username\

Contains user-specific information including:

- recently used files
- typed URLs
- application preferences
- mapped network drives
- executed commands.

This file is one of the **most valuable artifacts in forensic analysis**.

6. USRCLASS.DAT

Location:

C:\Users\Username\AppData\Local\Microsoft\Windows\

Contains:

- shellbag data
- user interface settings.

Shellbags store folder viewing preferences such as:

- folder location
- window size
- sort order.

These artifacts can reveal **folders that were accessed even if deleted later**.

8. Major Registry Hives Explained

1. HKEY_CLASSES_ROOT (HKCR)

Contains file extension associations.

Example:

.exe

.txt

.pdf

Also stores information related to:

- COM objects
- automation tools.

2. HKEY_CURRENT_USER (HKCU)

Contains configuration settings for the **currently logged-in user**.

Includes:

- desktop settings
- network connections
- printer configuration.

This hive is dynamically linked to the current user profile.

3. HKEY_LOCAL_MACHINE (HKLM)

Stores system-wide configuration data such as:

- hardware configuration
- operating system settings
- installed programs.

This hive contains most of the system configuration information.

4. HKEY_USERS (HKU)

Contains registry entries for **all user profiles on the system**.

Each user profile is identified by a **Security Identifier (SID)**.

Example:

S-1-5-21-xxxxxxx

5. HKEY_CURRENT_CONFIG (HKCC)

Stores information about the **current hardware configuration**.

Includes:

- display settings
- screen resolution
- hardware profiles.

This hive acts as a pointer to another registry location.

9. Registry Value Types

Registry values can store data in different formats.

Common types include:

Type	Description
REG_NONE	No defined type
REG_SZ	Text string
REG_EXPAND_SZ	String containing environment variables
REG_BINARY	Raw binary data
REG_DWORD	32-bit numeric value
REG_QWORD	64-bit numeric value
REG_MULTI_SZ	Multiple strings

These data types allow Windows to store various forms of configuration data.

10. Registry Artifacts in Forensics

The registry contains valuable forensic artifacts such as:

- program execution history
- USB device connections
- user activity traces
- installed software records
- startup programs.

Because of this, the registry is considered a **treasure trove of forensic evidence**.

11. Auto Startup Locations

Windows allows certain programs to run automatically during system startup.

Examples include:

- antivirus programs
- system services
- startup applications.

Malware can exploit this feature by inserting registry entries that allow it to **run automatically during system boot**.

Investigators check startup registry keys to detect persistence mechanisms used by attackers.

12. Volatile Registry Data

Some registry entries are **volatile**.

This means they exist only while the system is running.

For example:

HKEY_CURRENT_USER

This hive is generated when a user logs in and disappears when the system shuts down.

Therefore, investigators must capture volatile data while the system is still active.

13. Importance of the Windows Registry in Forensics

The registry is extremely valuable during forensic investigations because it provides:

1. Evidence of user activity
2. System configuration history
3. Malware persistence mechanisms
4. Installed software information
5. Device usage history.

Investigators often use registry analysis to **validate other digital evidence findings**.

Windows Event Logs

1. What are Event Logs?

Windows Event Logs record system events occurring within the operating system.

These logs help track:

- system errors
- user logins
- application activity
- security events.

Event logs are stored in XML format in:

System32\winevt\Logs

Registry settings controlling event logs are stored under:

HKLM\SYSTEM\CurrentControlSet\Services\EventLog

2. Types of Event Logs

1. Application Logs

These logs record events generated by applications installed on the system.

Example:

- application crashes
 - software errors.
-

2. Security Logs

Security logs record security-related activities such as:

- login attempts
- failed authentication
- privilege escalation.

These logs are essential in cybercrime investigations.

3. System Logs

System logs record events generated by the operating system.

Examples include:

- device driver errors
 - system startup and shutdown events.
-

4. Setup Logs

These logs record installation or upgrade events of the operating system.

5. Forwarded Events

These logs store events forwarded from other systems when centralized logging is enabled.

3. Event Log Severity Levels

Event logs are categorized into different levels:

Level	Meaning
Information	General information
Warning	Potential issue
Error	Application or system error
Critical	Serious failure
Success/Failure Audit	Security event logs

4. Event Log Fields

Each event log entry contains several fields.

Important fields include:

- Event ID
- Event Level
- Channel
- Computer Name
- Provider Name
- Timestamp

- Security User ID.

These fields help investigators identify specific events during analysis.

Memory Trick for Registry

To remember registry structure:

HKKSVD

H → Hive

K → Key

K → SubKey

S → Structure

V → Value

D → Data

If you want, I can also explain **Windows Registry artifacts investigators always check in exams and investigations**, such as:

- **Run / RunOnce keys**
- **USB device artifacts**
- **UserAssist keys**
- **ShimCache**
- **AmCache**

These are **very commonly asked forensic exam questions** and are extremely important.

Below is a **clear, detailed, exam-ready explanation of the exact content you shared**, expanded so you can **write a full 10-mark or 15-mark answer** and **remember the concepts easily**. I will explain each statement with **proper reasoning, examples, and forensic relevance**.

File Allocation Table (FAT) File System – Detailed Explanation

1. Introduction to FAT File System

The **File Allocation Table (FAT)** is one of the **oldest and simplest file systems** used by operating systems to store and organize files on storage devices.

A **file system** is responsible for:

- organizing data on a storage medium
- storing files and directories
- keeping track of file locations
- managing free and used disk space

FAT became widely popular because it is **simple to implement, lightweight, and compatible across multiple operating systems**.

Because of this simplicity, FAT is still used in many removable storage devices.

2. FAT in Operating Systems

Historically, FAT played an important role in early Microsoft operating systems.

FAT was the primary file system for:

- **MS-DOS**
- **Windows 95**
- **Windows 98**
- **Windows ME**

These systems used FAT because:

- it was easy to implement
- required minimal system resources
- worked well on small storage devices.

Later operating systems such as:

- Windows NT
- Windows 2000
- Windows XP

introduced a more advanced file system called **NTFS (New Technology File System)**.

NTFS provides advanced features such as:

- file permissions
- encryption
- journaling
- better reliability.

However, FAT remained widely used because it is **compatible with almost every operating system**, including:

- Windows
 - Linux
 - macOS
 - many Unix systems.
-

3. Where FAT is Commonly Used Today

Even though modern systems prefer NTFS or other file systems, FAT is still widely used in **removable storage devices**.

Examples include:

- USB flash drives
- SD cards
- memory cards
- digital camera storage
- embedded systems
- portable media players.

The main reason is **compatibility**.

Devices using FAT can easily be read by:

- computers
- cameras

- televisions
 - gaming consoles
 - mobile devices.
-

4. Basic Concept of the FAT File System

The fundamental idea of the FAT file system is that **every file and directory has a directory entry**.

A **directory entry** is a data structure that stores information about a file.

This information includes:

- file name
- file size
- starting cluster
- timestamps
- file attributes
- location of file data.

This structure helps the operating system locate the file on the disk.

Example:

File Name: report.doc

Size: 12 KB

Start Cluster: 8

The operating system reads the directory entry to locate the file's data.

5. Clusters – Storage Units of FAT

In FAT, files are stored in units called **clusters**.

A **cluster** is a group of one or more sectors.

Disk structure hierarchy:

Disk



Sector



Cluster



File Data

Example:

If

Sector size = 512 bytes

Cluster size = 4 sectors

Then Cluster size = $4 \times 512 = 2048$ bytes

Clusters are used because storing data in clusters improves disk management efficiency.

6. When a File Uses Multiple Clusters

If a file is larger than a single cluster, it must be stored in **multiple clusters**.

Example:

Cluster size = 4 KB

File size = 10 KB

The file will occupy:

Cluster 1 → 4 KB

Cluster 2 → 4 KB

Cluster 3 → 2 KB

However, the clusters might not be physically next to each other.

For example:

Cluster 5

Cluster 12

Cluster 30

Therefore, the system needs a way to **link clusters together**.

This is where the **File Allocation Table** becomes important.

7. Role of the FAT Structure

The **File Allocation Table** is a special table that keeps track of **cluster chains**.

The FAT table records:

- which clusters belong to which file
- which cluster comes next in a file
- which clusters are free
- which clusters are damaged.

Example:

Cluster 5 → points to Cluster 12

Cluster 12 → points to Cluster 30

Cluster 30 → End of File. This structure helps the system reconstruct the file.

8. FAT Stores Both Content and Metadata

The FAT structure helps in two ways:

1. Content Tracking

It helps locate the **next cluster of a file**, allowing the system to read the file sequentially.

2. Metadata Information

It also stores information about **cluster allocation status**, such as:

- free clusters
- used clusters
- bad clusters
- end-of-file markers.

Because of this, FAT is used for both:

- **data management**
 - **metadata management.**
-

9. Types of FAT File Systems

There are **three main versions of FAT**.

FAT12

- Uses **12-bit cluster addresses**
- Used mainly in **floppy disks**
- Supports small storage sizes.

FAT16

- Uses **16-bit cluster addresses**
- Used in early hard drives
- Supports larger volumes than FAT12.

FAT32

- Uses **32-bit cluster addresses**
 - Supports much larger storage devices
 - Most common FAT version used today.
-

Major Difference Between FAT Versions

The main difference between FAT12, FAT16, and FAT32 is:

size of entries in the FAT table.

Example:

FAT Type	Entry Size	Max Storage
FAT12	12 bits	~32 MB
FAT16	16 bits	~2 GB
FAT32	32 bits	up to 2 TB

Larger entries allow more clusters to be addressed.

10. Boot Sector in FAT File System

The **boot sector** is a critical structure in the FAT file system.

It is located in:

First sector of the volume

The boot sector contains essential information such as:

- sector size
- cluster size
- number of FAT tables
- reserved sectors
- file system parameters.

This information helps the operating system understand how the disk is organized.

The boot sector belongs to the **reserved area** of the FAT file system.

11. FAT Type Identification

Interestingly, the FAT file system **does not contain a direct field indicating whether it is FAT12, FAT16, or FAT32.**

Instead, the type is determined by **calculating the number of clusters in the file system** using values from the boot sector.

Based on the cluster count, the file system type is identified.

Example logic:

Cluster Count < 4085 → FAT12

Cluster Count < 65525 → FAT16

Cluster Count ≥ 65525 → FAT32

12. Additional Features of FAT32

FAT32 introduces additional structures to improve reliability.

Backup Boot Sector

FAT32 stores a **backup copy of the boot sector.**

Location: Sector 6

Purpose: used if the main boot sector becomes corrupted.

Tools can automatically recover the boot sector using this backup.

13. FSINFO Structure

FAT32 also contains an **FSINFO data structure**.

FSINFO stores:

- number of free clusters
- location of the next available cluster.

This allows the system to quickly find free disk space without scanning the entire disk.

14. Layout of FAT File System

The FAT file system is divided into **three physical sections**.

1. Reserved Area

This area contains file system metadata such as:

- boot sector
- BIOS parameter block
- FSINFO structure (in FAT32)
- backup boot sector.

In FAT12 and FAT16 this area is usually **1 sector**, but the actual size is specified in the boot sector.

2. FAT Area

The FAT area stores:

- primary FAT table
- backup FAT table.

These tables maintain the **cluster allocation information**. The FAT area begins immediately after the reserved area.

3. Data Area

The data area contains:

- file data
- directory structures
- clusters allocated to files.

All files and directories are stored inside clusters located in this area.

Why FAT is Important in Digital Forensics

FAT file systems are extremely important in forensic investigations because:

- many removable devices use FAT
- deleted files can often be recovered
- cluster chains reveal file storage patterns
- directory entries contain timestamps and metadata.

Investigators often analyze:

- directory entries
- FAT tables
- cluster chains
- deleted file indicators.

Below are **complete, detailed notes from Page 1 to Page 80** of the NTFS unit in an **exam-oriented structure**, written so you can **write 10–15 mark answers and remember easily**. I expanded the points while keeping the **exact concepts from the material**.

UNIT 3 — NTFS (New Technology File System)

1. Introduction to NTFS

Definition

NTFS (New Technology File System) is a **file system developed by Microsoft** that is used by Windows operating systems to **store, organize, and manage files on storage devices**.

A file system determines:

- how files are stored
 - how files are organized
 - how free space is managed
 - how files are retrieved.
-

History of NTFS

NTFS was introduced with:

Windows NT 3.1 in 1993.

Since then it has become the **default file system** for modern Windows operating systems such as:

- Windows 2000
 - Windows XP
 - Windows Vista
 - Windows 7
 - Windows 8
 - Windows 10
 - Windows 11.
-

Why NTFS Was Developed

Before NTFS, Windows used the **FAT file system**.

However, FAT had several limitations:

- limited file size
- poor security
- no access control
- high fragmentation
- weak reliability.

NTFS was designed to overcome these limitations.

2. Key Features of NTFS

NTFS provides several advanced features.

1. Improved Security

NTFS supports:

- file permissions
- access control lists (ACL)
- user authentication.

This allows administrators to control:

- who can read files
 - who can write files
 - who can execute files.
-

2. Encryption

NTFS supports **Encrypting File System (EFS)**.

This allows files to be encrypted so that **unauthorized users cannot access them**.

3. Compression

NTFS allows files to be stored in **compressed format** to save disk space.

Files are automatically decompressed when accessed.

4. Disk Quotas

Disk quotas allow administrators to:

- limit the amount of disk space a user can use.

Example:

User A → 5 GB limit.

5. Journaling

NTFS uses **journaling**.

Journaling records changes made to the file system.

If the system crashes, NTFS can recover using the journal log.

This helps prevent **data corruption and file loss**.

6. Large File Support

NTFS supports extremely large storage.

Maximum limits:

Feature	Limit
Maximum File Size	16 Exabytes

Maximum Volume Size 256
Terabytes

Actual limits depend on hardware and operating system.

7. Links Support

NTFS supports:

- **hard links**
- **symbolic links**

These allow files to reference the same data location.

8. Reliable Metadata Storage

NTFS stores metadata in a structure called the **Master File Table (MFT)**.

This allows:

- faster file lookup
 - better organization.
-

3. What is MFT (Master File Table)

The **Master File Table (MFT)** is the **core structure of NTFS**.

It acts as a **database that stores information about every file and directory on the disk**.

Every file stored on an NTFS volume has an entry in the MFT.

Location of MFT

The MFT is stored at a **fixed location on the disk**.

Advantages:

- faster file access
 - easy indexing
 - quick lookup.
-

Growth of MFT

The MFT is created when:

the volume is formatted with NTFS

As new files are added:

new records are added to the MFT

Therefore the MFT **grows dynamically**.

4. Structure of the MFT

The MFT is composed of **records**.

Each record represents:

- a file
- a directory.

Each record contains **attributes describing the file**.

MFT Record Structure

Each MFT record contains:

- header

- attributes.
-

MFT Record Header

The header stores:

- record identifier
 - attribute location
 - record status
 - reference number.
-

Attributes Section

Attributes describe the file.

Examples:

- file name
 - file size
 - permissions
 - timestamps.
-

5. MFT Records

Definition

MFT records are the **basic building blocks of NTFS**.

Each record corresponds to **one file or directory**.

First Record of MFT

The **first record** is called:

Master File Record (MFR)

This record stores information about the MFT itself.

File Reference Number (FRN)

Each MFT record has a unique identifier called the:

File Reference Number (FRN)

This number is used to reference files within the system.

Sequence Number

Each MFT entry contains:

16-bit sequence number

This number increases when the entry is reused.

The sequence number helps detect:

- corruption
 - invalid references.
-

File Reference Address

The combination of:

MFT Entry + Sequence Number

forms a **64-bit file reference address**.

This helps NTFS track files safely.

6. Attributes in MFT Records

Attributes store file metadata.

Attributes can be classified into two types.

1. Resident Attributes

Resident attributes are stored **directly inside the MFT record**.

Example:

small files.

2. Non-Resident Attributes

Non-resident attributes are stored **outside the MFT record**.

The MFT record only contains pointers to the data.

Example:

large files.

Non-resident attributes are stored in cluster runs, which are consecutive clusters, and the run is documented using the starting cluster address and run length.

- NTFS uses VCN-to-LCN mappings to describe the non-resident attribute runs.
- Logical Cluster Number (LCN) is the same as the logical file system address.
- Virtual Cluster Number (VCN) is the same as a logical file address.

Attribute Structure

Attributes consist of:

Header

Contains:

- attribute type
- attribute length
- attribute flags.

Body

Contains the **actual attribute data**.

7. Default MFT Attributes

Each file normally contains the following attributes.

1. \$STANDARD_INFORMATION

Stores basic file metadata such as:

- creation time
- modification time
- last access time
- file permissions.

2. \$FILE_NAME

Stores:

- file name
- parent directory
- creation timestamp.

3. \$OBJECT_ID

Stores a unique identifier for the file.

This allows files to be tracked across volumes.

4. \$SECURITY_DESCRIPTOR

Contains security information such as:

- file owner
- group information
- access control list (ACL).

5. \$VOLUME_NAME

Stores the name of the disk volume.

Example:

C:

DATA

BACKUP

6. \$VOLUME_INFORMATION

Stores information about the volume such as:

- creation time
- volume serial number
- number of sectors.

7. \$DATA

Stores the **actual content of the file**.

Examples:

- text files
- program binaries
- images.

Small files may be **stored entirely inside the MFT record**.

Large files are stored in clusters outside the MFT.

8. NTFS Metadata Files

NTFS reserves the **first 16 MFT entries** for system metadata files.

These files are hidden and start with \$.

Example:

\$MFT

\$Bitmap

\$LogFile

These files manage the NTFS file system itself.

Important NTFS Metadata Files

\$MFT

Contains the Master File Table.

\$MFTMirr

Backup copy of the MFT.

Used if the main MFT becomes corrupted.

\$LogFile

Stores the NTFS **transaction log**.

Records changes made to the file system.

\$Volume

Stores information about the volume.

\$AttrDef

Contains definitions for file attributes.

\$Bitmap

Stores a bitmap of:

- used clusters
 - free clusters.
-

\$Boot

Stores the boot sector of the NTFS volume.

\$BadClus

Stores information about bad clusters.

\$Secure

Stores security policies and permissions.

9. Benefits of MFT

Advantages of using the Master File Table include:

1. Fast File Access

The fixed location of the MFT allows quick lookup.

2. Reduced Fragmentation

File metadata is stored in a continuous block.

3. Advanced Features

Supports:

- encryption
 - compression
 - access control.
-

10. Drawbacks of MFT

Although powerful, the MFT has some disadvantages.

1. Fragmentation

If the disk becomes full, the MFT may fragment.

2. Corruption Risk

If the MFT becomes corrupted:

- the file system may become inaccessible.
-

3. Compatibility Issues

NTFS is mainly supported by Windows.

Other operating systems may have limited support.

11. NTFS in Digital Forensics

NTFS is extremely important in forensic investigations.

Because it stores **rich metadata**, investigators can analyze:

- timestamps
 - file history
 - permissions
 - file attributes.
-

12. NTFS Header Analysis

The NTFS header is located in:

sector 0

of the NTFS volume.

The header contains information such as:

- sectors per cluster
 - MFT size
 - MFT mirror location.
-

Why Header Analysis is Important

Header analysis helps investigators:

- understand disk structure
 - detect tampering
 - locate MFT and other structures.
-

Information Extracted from NTFS Header

Investigators can extract:

- cluster size
- volume layout

- boot sector data
 - MFT mirror location.
-

Detecting Tampering

Incorrect header values may indicate:

- disk corruption
 - malware activity
 - intentional manipulation.
-

13. MFT Forensic Analysis

The MFT provides investigators with valuable information.

It stores:

- file name
 - file size
 - timestamps
 - file attributes
 - file location.
-

Recovering Deleted Files

Deleted file entries may still exist in the MFT.

Forensic tools analyze these entries to:

- recover deleted files
 - reconstruct activity timelines.
-

Information Extracted from MFT

Investigators can obtain:

File Names

Includes deleted files.

File Attributes

Example:

- hidden
 - read-only
 - encrypted.
-

File Timestamps

Includes:

- creation time
 - modification time
 - access time.
-

File Size

Stored in MFT entry.

File Location

Starting cluster of the file.

14. \$LogFile Analysis

The **\$LogFile** records changes made to the file system.

Examples:

- file creation
 - file deletion
 - file modification.
-

Structure of \$LogFile

The log is a **circular log**.

This means:

- new entries overwrite old entries when full.
-

Importance in Forensics

\$LogFile analysis helps investigators determine:

- sequence of file operations
 - timeline of user activity
 - potential malware behavior.
-

Evidence from \$LogFile

Investigators can find:

- file paths
 - timestamps
 - user actions.
-

15. Tools Used in NTFS Forensics

Common forensic tools include:

- FTK Imager

- EnCase
- Autopsy
- Sleuth Kit
- dd (disk imaging).

These tools help analyze:

- MFT
 - \$LogFile
 - \$Bitmap
 - metadata.
-

16. Challenges in NTFS Forensics

NTFS analysis can be difficult because:

- the structure is complex
 - attackers may hide data
 - logs may overwrite older entries.
-

17. NTFS Investigations are Used In

NTFS forensic analysis is used in cases such as:

- cybercrime investigations
 - intellectual property theft
 - financial fraud
 - insider threats
 - child exploitation cases.
-

Quick Exam Memory Summary

NTFS Key Components

NTFS

- └ Boot Sector
 - └ MFT
 - └ MFT Mirror
 - └ \$LogFile
 - └ \$Bitmap
 - └ Data Clusters
-

If you want, I can also give you **extremely important exam questions from this unit (that professors repeatedly ask)** such as:

- **Explain MFT structure with diagram**
- **Explain NTFS metadata files**
- **Explain NTFS forensic analysis**
- **Difference between FAT and NTFS**

These questions almost always appear in **digital forensics exams**.

Sans

SANS Incident Handling Steps

The **SANS Institute** recommends **six steps for effective incident handling** in cybersecurity. These steps ensure that security incidents are detected, contained, resolved, and analyzed properly.

The six steps are:

1. Preparation
2. Identification
3. Containment
4. Eradication
5. Recovery
6. Lessons Learned

These steps together form the **Incident Response Lifecycle** used by SOC teams, CERT teams, and forensic investigators.

1. Preparation (Preventing Incidents)

Preparation is the **most important phase** because it ensures that an organization is **ready to detect and respond to attacks quickly**.

The main goal is to **reduce the chances of incidents and improve response capability**.

Key Activities in Preparation

1. Risk Assessment

Organizations must identify possible risks and vulnerabilities.

Example:

- Weak passwords
- Unpatched servers
- Open network ports

Example scenario:

A company discovers through risk assessment that **outdated web servers are vulnerable to SQL injection attacks**.

2. Host Security

Security controls should be implemented on individual systems.

Examples:

- Endpoint protection
- Patch management
- Secure configurations
- Access control policies

Example: Installing **EDR (Endpoint Detection and Response)** tools on employee laptops.

3. Network Security

Protect the network infrastructure.

Techniques include:

- Firewalls
- IDS/IPS
- Network segmentation
- VPN security

Example: A company deploys **Snort IDS** to detect malicious traffic.

4. Malware Prevention

Measures should be implemented to detect and prevent malware infections.

Examples:

- Antivirus software
- Sandboxing
- Email filtering
- Web filtering

Example: Blocking malicious attachments in email gateways.

5. User Awareness & Training

Employees should be trained about cybersecurity threats.

Training topics include:

- Phishing attacks
- Password security

- Social engineering
- Data protection

Example:

An employee identifies a phishing email because of prior training and reports it.

2. Identification (Detecting an Incident)

Identification involves **detecting and confirming that an incident has occurred**.

An organization must determine:

- Whether an incident is real
- How severe it is
- Which systems are affected

Attack Vectors (Ways an Attack Enters)

According to SANS, incidents may originate from several attack vectors.

1. External or Removable Media

Malware enters through USB drives or external devices.

Example: An employee inserts a **malware-infected USB** which installs ransomware.

2. Web

Users visiting malicious websites may download malware.

Example: A fake software update site installs spyware.

3. Email

Phishing emails can deliver malware or steal credentials.

Example: An email pretending to be from HR asks employees to reset passwords.

4. Impersonation

Attackers pretend to be trusted individuals.

Example: An attacker impersonates the IT department requesting login credentials.

5. Improper Usage

Employees misuse systems.

Example: Installing unauthorized software.

6. Loss or Theft of Equipment

Devices containing sensitive data may be stolen.

Example: A stolen laptop contains confidential company documents.

Sources of Indicators and Precursors

Indicators help security teams detect incidents.

1. Security Alerts

Security tools generate alerts.

Examples:

- Firewall alerts
- Security monitoring alerts

2. IDS and IPS

Intrusion Detection Systems and Intrusion Prevention Systems identify suspicious activity.

Example: Snort detecting port scanning attempts.

3. SIEM Systems

Security Information and Event Management tools analyze logs from multiple sources.

Example: Splunk detecting unusual login patterns.

4. Antivirus Alerts

Malware detection on endpoints.

Example: Windows Defender detecting trojan activity.

5. File Integrity Checking Software

Detects unauthorized changes in files.

Example: Tripwire detecting modified system files.

6. Third-Party Monitoring Services

External security providers may detect attacks.

Example: Cloud security provider reports suspicious API usage.

Logs as Indicators

Logs provide critical forensic evidence.

Types of logs include:

1. OS Logs

System level activities.

Example:

Login failures recorded in Windows Event Logs.

2. Service Logs

Logs generated by services like databases or web servers.

Example: Apache logs showing suspicious requests.

3. Application Logs

Logs from applications.

Example: Payment application logging failed transactions.

4. Network Logs

Traffic logs from routers and firewalls.

Example: Firewall logs showing unusual outbound traffic.

Public Information Sources

Information about threats can come from external sources.

Examples:

- Newly discovered vulnerabilities
- Security advisories
- Threat intelligence feeds

Human Sources

Information may come from people.

Examples:

- Employees reporting suspicious activity
- Security researchers reporting vulnerabilities.

3. Incident Analysis

After detecting a possible incident, analysts must **verify and investigate it**.

However, incident analysis is difficult because:

- Indicators may be false positives
- Systems generate thousands of alerts daily.

Example:

An IDS alert might incorrectly flag normal traffic as malicious.

Initial Analysis Activities

1. Profiling Systems and Networks

Understand how systems normally behave.

Example:

Typical login times for employees.

2. Understanding Normal Behavior

Baseline behavior helps identify anomalies.

Example:

A server normally sends 10MB traffic but suddenly sends 5GB.

3. Log Retention Policy

Organizations must store logs for analysis.

Example: Keeping logs for 6 months for forensic investigation.

4. Event Correlation

Combining data from multiple sources to detect patterns.

Example: Firewall logs + login logs showing brute force attack.

5. Time Synchronization

All systems should have synchronized clocks.

Example:

Using **NTP (Network Time Protocol)**.

This helps correlate logs accurately.

Recommendations for Initial Analysis

SANS suggests:

1. Maintain a knowledge base of known attacks
2. Use search engines for research
3. Use packet sniffers to capture network traffic
4. Filter irrelevant data
5. Seek assistance from experts

Example:

Using **Wireshark** to capture suspicious network packets.

Incident Documentation

All details must be documented once an incident is suspected.

Documentation helps in:

- investigation
- legal proceedings
- improving future responses

Information Recorded

1. Incident status
2. Summary of the incident
3. Indicators related to the attack
4. Related incidents
5. Actions taken by incident handlers
6. Chain of custody (for evidence)
7. Impact assessment
8. Contact information of involved parties
9. Evidence collected
10. Comments from investigators
11. Next steps

Example:

A logbook recording each action taken during a ransomware attack investigation.

Incident Prioritization

Incidents must be prioritized.

Important rule:

 **Incidents should never be handled on a first-come first-served basis.**

They must be prioritized based on:

1. Functional Impact
2. Information Impact
3. Recoverability

Functional Impact

Measures how the incident affects system operations.

Levels include:

Category	Meaning
None	No effect on services

Low	Minor effect
Medium	Some users lose critical services
High	Critical services unavailable

Example: A banking server outage affecting all users = **High impact**.

Information Impact

Measures damage to information.

Category	Meaning
None	No data compromised
Privacy Breach	Personal data leaked
Proprietary Breach	Sensitive business data leaked
Integrity Loss	Data altered or deleted

Example: Customer credit card data leaked.

Recoverability

Measures how easily systems can be restored.

Category	Meaning
Regular	Recovery easy
Supplemented	Needs additional resources
Extended	Recovery unpredictable
Not Recoverable	Permanent damage

Example:
Ransomware encrypting backups = **Not recoverable**.

Incident Notification

After analysis, the incident must be reported.

People involved may include:

- CIO
- Head of Information Security
- Local Security Officer
- Other incident response teams
- System owners
- HR department
- Legal department
- Public relations
- CERT-In
- Law enforcement.

Communication Methods

Possible communication channels:

- Email
- Internal website portals
- Phone calls
- In-person briefings
- Voice mailbox updates
- Paper notices.

3. Containment

Containment aims to **limit the damage caused by an incident**.

It prevents the attack from spreading.

Example:

Disconnecting an infected machine from the network.

Factors in Choosing Containment Strategy

1. Potential damage
2. Need to preserve evidence

3. Service availability
4. Time required
5. Effectiveness of the strategy
6. Duration of containment solution

Example:

Blocking malicious IP addresses in firewall.

Evidence Gathering

Evidence must be collected carefully.

Important principles:

- Collect evidence as soon as possible
- Take a **system image** before investigation
- Preserve data integrity

Example:

Creating a **forensic disk image** using **FTK Imager**.

Identifying the Attacker

Steps include:

1. Verify attacker IP address
2. Research IP using search engines
3. Use incident databases
4. Monitor attacker communication channels.

Example:

Using **WHOIS lookup** to identify attacker origin.

4. Eradication

Eradication removes the **root cause of the attack**.

Actions include:

1. Removing malware
2. Disabling compromised accounts

3. Fixing vulnerabilities
4. Patching systems
5. Identifying all affected systems.

Example:

Deleting ransomware and patching exploited vulnerabilities.

5. Recovery

Recovery restores systems to **normal operational state**.

Actions include:

1. Restoring from clean backups
2. Rebuilding systems
3. Replacing compromised files
4. Installing patches
5. Changing passwords
6. Strengthening network security.

Example:

Restoring a database from a backup after malware infection.

6. Lessons Learned (Post Incident Activity)

After incident resolution, organizations analyze the incident.

Purpose:

- Improve security
- Prevent similar attacks.

Organizations conduct a **Lessons Learned Meeting**.

Questions Asked During Review

1. What exactly happened?
2. How well did staff handle the incident?
3. Were procedures followed correctly?
4. What information was missing?

5. Did any action delay recovery?
6. What should be done differently next time?
7. What new indicators should be monitored?
8. What tools are required in the future?

Incident Metrics

Collected data helps measure response effectiveness.

Examples:

- Number of incidents handled
- Time spent per incident
- Response time
- Total labor involved
- Detection time.

Evidence Retention

Organizations must define how long evidence should be stored.

Factors considered:

1. Legal prosecution
2. Data retention policies
3. Storage costs.

Example: Logs and forensic images stored for **2–5 years**.

Goals of Incident Response:

1. Verify that an incident has occurred.
2. Maintain or restore business continuity.
3. Reduce the impact of the incident.
4. Determine how the attack happened.
5. Prevent future attacks.
6. Improve overall security and response capability.
7. Support legal prosecution of attackers.
8. Keep management informed about the situation.

No.	Precursor	Indicator
1	A precursor is a sign that an attack may occur in the future.	An indicator is evidence that an attack is occurring or has occurred.
2	It represents a warning signal before a security incident.	It represents proof or confirmation of a security incident.
3	It helps predict potential threats.	It helps detect active or completed attacks.
4	It occurs before the actual incident.	It occurs during or after the incident.
5	It indicates possible malicious activity.	It indicates confirmed malicious activity.
6	It is usually less reliable and may not lead to an attack.	It is more reliable evidence of compromise.
7	It helps security teams prepare defenses in advance.	It helps security teams respond to incidents immediately.
8	Often detected through threat intelligence or reconnaissance activity.	Often detected through security alerts or system logs.
9	It suggests attack planning or reconnaissance by attackers.	It shows actual exploitation or unauthorized activity.
10	Used mainly in early threat detection and prevention.	Used mainly in incident detection and investigation.
11	May not require immediate incident response.	Usually requires immediate incident response actions.
12	Helps organizations strengthen security controls beforehand.	Helps organizations contain and mitigate attacks.
13	Examples include port scanning, vulnerability scanning, or threat intelligence reports.	Examples include malware detection, unauthorized login, or data exfiltration.
14	Indicates potential risk.	Indicates actual compromise or attack activity.
15	Used for proactive security monitoring.	Used for reactive incident response and investigation.

IR

Basis	Precursors	Indicators
1. Definition	Signs that an attack may happen in future	Evidence that an attack is happening or has happened
2. Timing	Before an incident	During or after an incident
3. Nature	Predictive / warning signals	Confirmatory / evidential signals
4. Purpose	Helps in preventing attacks	Helps in detecting and responding to attacks
5. Certainty Level	Low certainty (possibility)	High certainty (strong evidence)
6. Example Type	Threat intelligence alerts, vulnerability reports	Log anomalies, malware detection, unusual traffic
7. Source	External intel, risk assessments, scanning tools	IDS/IPS alerts, SIEM logs, system monitoring
8. Action Taken	Proactive measures (patching, hardening)	Reactive measures (containment, eradication)
9. Role in IR	Part of early warning phase	Part of incident detection & analysis phase
10. Example	Info about planned phishing campaign	Multiple failed logins + unauthorized access

Unit 4

Disadvantages of Digital Forensics

1. Evidence must be proven untampered.
2. Digital evidence acceptance is strict.
3. Storage and processing is costly.
4. Requires expert knowledge.
5. Needs highly skilled investigators.
6. Requires proper tools.
7. Invalid tools can reject evidence in court.
8. Legal practitioners must understand technology.
9. Lack of knowledge can lead to wrong results.

Advantages of Digital Forensics

1. Ensures system integrity.
2. Produces valid court evidence.
3. Helps punish criminals.
4. Helps organizations recover data after attack.
5. Tracks cybercriminals globally.
6. Saves money and time.
7. Extracts factual evidence.
8. Helps interpret evidence correctly.
9. Strengthens legal cases.

1. Data Carving

Definition and Core Idea

1. Data carving is the process of searching and extracting specific data patterns (strings or bytes) from raw storage.
2. It does not depend on file system structure like file tables.
3. It directly scans raw data such as disk sectors or memory.

4. It is mainly used when files are deleted or metadata is missing.

How It Works

5. The investigator identifies a pattern such as file signature (header/footer).
6. Example: JPG files start with **FFD8** and end with **FFD9**.
7. Tools scan entire storage for such patterns.
8. Once found, data between header and footer is extracted.

Tools and Techniques

9. Tools include hex editors, Scalpel, Foremost.
10. Can search using ASCII strings or binary patterns.
11. Works on multiple targets like full disk, RAM, swap file.
12. Also works on slack space and unallocated space (as shown on page 7).

Types of Carving

13. File carving recovers complete files using signatures.
14. Record carving extracts partial data like logs or entries.
15. Useful when full file is not required but information is important (page 10).

Limitations

16. Works best for contiguous (non-fragmented) files (page 9).
17. Fragmented files are harder to reconstruct.
18. Advanced methods like fuzzy hashing can help.

Importance

19. Helps recover deleted evidence intentionally removed.
20. Works even when file system is damaged.

Example of Data Carving (from PDF)

21. A forensic investigator received a seized system (page 11).
22. The system logs were deleted.
23. Very little logical evidence was available.
24. Investigator performed data carving on raw storage.
25. Recovered 22,000+ Windows Event Log entries.
26. Extracted details like IP address, hostname, timestamp (page 12).
27. These logs proved malware communication and intrusion.

This shows carving can recover hidden or deleted evidence.

2. Data Acquisition (Detailed Explanation)

Definition and Core Idea

1. Data acquisition is the process of collecting digital evidence.
2. It is the first step in digital forensic investigation.
3. It ensures evidence is captured in a forensically sound manner.
4. It includes both live data and static data collection (page 13).

Types of Data Acquisition

5. Live acquisition includes RAM capture, running processes.
6. Static acquisition includes disk imaging of powered-off systems.
7. Both types are important depending on investigation need.

Key Principle

8. Must create minimum changes to the original system.
9. Integrity of evidence must be preserved.
10. Original data should never be modified.

Acquisition Process (from PDF steps)

11. System is powered off for safe acquisition (page 14).
12. Hard drive is removed from system.
13. Connected to a write blocker (read-only device).
14. Write blocker prevents modification of original data.
15. A target storage device is connected to store copy.
16. System is connected to analysis machine.
17. Bit-by-bit image (forensic copy) is created (page 15).
18. Original evidence is preserved and documented.
19. Chain of custody is maintained.
20. Image is used for further analysis instead of original.
21. Analysis is done in read-only mode (page 16).
22. Multiple copies can be created safely.
23. Ensures repeatability of investigation.

Importance

- 24. Ensures legal admissibility of evidence.
- 25. Protects against accidental data modification.
- 26. Provides exact replica for analysis.

Example of Data Acquisition

- 27. Suppose police seize a suspect's laptop.
- 28. Investigator removes the hard disk.
- 29. Connects it via write blocker.
- 30. Creates a forensic image using FTK Imager.
- 31. Original disk is sealed and stored.
- 32. All analysis is done on copied image.

This ensures evidence remains unchanged and valid in court.

No.	Data Acquisition	Data Carving
1	Process of collecting digital evidence from a device	Process of recovering deleted or hidden files from raw data
2	Done at the initial stage of investigation	Done during analysis phase
3	Focuses on capturing complete data from storage	Focuses on extracting specific files or fragments
4	Creates an exact copy (image) of storage media	Reconstructs files without filesystem metadata
5	Includes disk imaging, memory acquisition	Includes file recovery techniques
6	Works on active and intact data	Works mainly on deleted or corrupted data
7	Uses tools like FTK Imager, EnCase	Uses tools like Scalpel, Foremost
8	Maintains original evidence integrity	Attempts to recover lost evidence
9	Requires write blockers to avoid modification	Does not require direct device interaction

10	Produces a forensic image (bit-by-bit copy)	Produces recovered files (partial or full)
11	Based on system structure and storage layout	Based on file signatures (headers/footers)
12	Ensures data preservation for investigation	Helps in retrieving hidden evidence
13	No modification is allowed during process	Reconstruction may not always be perfect
14	Legally critical step for evidence collection	Supporting step for evidence discovery
15	Output is complete dataset for analysis	Output is selected recovered data

1. Chain of Custody

1. Chain of custody is the complete record of how digital evidence is handled from collection to court.
 2. It tracks every movement of evidence from one person or place to another.
 3. It begins when evidence is first collected from the crime scene.
 4. Evidence is logged when it enters the forensic lab.
 5. It is stored securely to prevent tampering.
 6. When analysis is required, it is checked out to an examiner.
 7. After analysis, it is returned and logged again.
 8. Each transfer must include date, time, and responsible person.
 9. Even minor handling must be documented.
 10. It ensures transparency in the investigation process.
 11. It prevents unauthorized access or modification.
 12. Courts require proper documentation to accept evidence.
 13. Missing records can make evidence unreliable.
 14. If broken, the evidence may be rejected as inadmissible.
 15. Overall, it ensures trust, integrity, and legal validity of evidence.
-

2. Hash Value

1. A hash value is a fixed-length unique output generated from data.
2. It is created using cryptographic hashing algorithms.

3. It acts like a digital fingerprint of data.
 4. Even a single bit change alters the hash completely.
 5. It is used to verify data integrity.
 6. Common algorithms include MD5, SHA-1, and SHA-2.
 7. Same input always produces the same hash output.
 8. Different inputs ideally produce different hashes.
 9. Used widely in digital forensics and cryptography.
 10. Helps detect tampering or modification of data.
 11. Used during disk imaging and evidence collection.
 12. Ensures original data remains unchanged.
 13. Quick way to compare large files without reading them fully.
 14. Considered reliable for verification purposes.
 15. Weak algorithms may suffer from collisions.
-

3. Use of Hash Value

1. Hash values are used throughout the forensic process.
 2. They verify integrity of collected evidence.
 3. Used immediately after disk cloning.
 4. Confirms clone is an exact copy of original.
 5. Helps ensure no data loss during duplication.
 6. Used repeatedly at different investigation stages.
 7. Acts as an integrity checkpoint.
 8. Useful when sharing forensic images between investigators.
 9. Receiver compares hash with original value.
 10. Matching hashes confirm authenticity.
 11. Ensures evidence is unchanged during transfer.
 12. Hash values are documented in reports.
 13. These records are presented in court.
 14. Helps prove evidence reliability before jury.
 15. Essential for maintaining credibility of digital evidence.
-

4. File Hashing

1. File hashing uniquely identifies a file using a hash value.
2. It converts file content into a fixed-length string.
3. Different algorithms can be used for hashing.
4. Strong algorithms reduce chances of collisions.
5. Weak algorithms may produce same hash for different files.

6. Used to compare files quickly.
 7. Helps identify duplicate or known files.
 8. Common in incident response investigations.
 9. Analysts search hash values online for identification.
 10. Helps determine file behavior or origin.
 11. Used in malware detection and classification.
 12. In contraband cases, hashes match known illegal content.
 13. Helps law enforcement identify known victims or materials.
 14. Used for whitelisting trusted files.
 15. Reduces data volume by filtering irrelevant files.
-

5. File Hashing (Advanced Use / Continuation)

1. Matching hashes confirms if file was seen before.
2. Useful in identifying known malicious files.
3. Helps avoid repeated analysis of same file.
4. Supports database-based investigations.
5. Used in child abuse material detection cases.
6. Enables quick verification against known datasets.
7. Improves investigation speed and efficiency.
8. Used in digital evidence categorization.
9. Helps prioritize suspicious files.
10. Whitelisting removes known safe files from analysis.
11. Reduces workload of forensic analysts.
12. However, attackers may misuse trusted files.
13. Legitimate system files can be abused in attacks.
14. Over-filtering may hide important evidence.
15. Requires careful balance between filtering and analysis.

No.	Data Recovery	Data Carving
1	Process of restoring lost, deleted, or inaccessible data	Process of extracting data from raw storage using patterns/signatures
2	Relies on file system metadata (FAT, MFT, inodes)	Does not rely on file system metadata
3	Works when file system is partially intact	Works even when file system is completely damaged

4	Recovers full files with original structure (name, path, timestamps)	Usually recovers files without original metadata
5	Uses logical structure of storage	Uses raw data scanning (byte-level analysis)
6	Faster if metadata is available	Slower due to deep scanning of entire storage
7	More accurate reconstruction of files	May produce incomplete or fragmented files
8	Handles deleted files using file system references	Handles deleted/hidden data using file signatures
9	Tools include Recuva, EaseUS, Disk Drill	Tools include Foremost, Scalpel, PhotoRec
10	Suitable for accidental deletion cases	Suitable for forensic investigations and hidden data
11	Limited when metadata is overwritten	Still works even if metadata is overwritten
12	Recovers file hierarchy (folders, structure)	Cannot recover folder structure
13	Less technical complexity	More complex and requires forensic expertise
14	Used in general data restoration scenarios	Used in digital forensics and incident response
15	Output is organized, usable files	Output may require manual analysis and reconstruction

Types of Data Carving Methods

1. Header–Footer (Signature-Based) Carving

This is the most common method. It searches for known file signatures (headers and footers) in raw data.

How it works:

- Each file type has a unique starting and ending pattern
- Tool scans disk for header
- Continues until it finds matching footer
- Extracts data in between

Example: JPG file starts with **FFD8** and ends with **FFD9**, so carving tools recover images using these markers.

Key point: Works best for non-fragmented (contiguous) files.

2. Header–Size Carving

Instead of searching for footer, this method uses file size information stored in header.

How it works:

- Detect header
- Read size field inside header
- Extract exact number of bytes

Example: BMP or some document formats store size in header, so tool uses that to extract file.

Key point: Useful when footer is missing or not clearly defined.

3. File Structure-Based Carving

Uses internal structure of file instead of only header/footer.

How it works:

- Understands format of file (like PDF, DOCX)
- Parses internal fields and structure
- Extracts valid data segments

Example: Recovering a PDF by analyzing its object structure.

Key point: More accurate than simple signature carving.

4. Content-Based Carving

Searches for specific content or keywords instead of file structure.

How it works:

- Looks for text patterns or strings
- Extracts surrounding data

Example: Searching for keyword "password" or email IDs in raw disk.

Key point: Useful for record-level recovery, not full files.

5. Block-Based (Fragment-Based) Carving

Works when files are fragmented across disk.

How it works:

- Identifies blocks belonging to same file
- Reassembles them using patterns or similarity

Example: Recovering a fragmented video file split across disk sectors.

Key point: More complex and less accurate than contiguous carving.

6. Smart / Context-Aware Carving

Uses intelligent algorithms and heuristics.

How it works:

- Combines multiple techniques
- Uses file format knowledge + behavior patterns
- May use AI or statistical methods

Example: Advanced forensic tools reconstruct partially corrupted files.

Key point: Handles complex and fragmented data better.

7. Fuzzy Hashing-Based Carving

Uses similarity matching instead of exact signatures.

How it works:

- Compares unknown data with known file patterns
- Finds approximate matches

Example: Detecting modified malware samples similar to known ones.

Key point: Useful when a file is slightly changed or obfuscated.

8. Metadata-Based Carving (Limited Use)

Uses any remaining metadata fragments.

How it works:

- Extracts partial file system information
- Combines with carving

Example: Recovering file fragments using leftover inode or MFT entries.

Key point: Hybrid approach between recovery and carving.

Quick Summary

- Signature-based → simple and fast
- Structure-based → accurate
- Content-based → keyword focused
- Fragment-based → handles broken files
- Smart/fuzzy → advanced reconstruction

1. What is a Write Blocker

A write blocker is a hardware or software tool used in digital forensics to prevent any modification to the original storage device during data acquisition or analysis.

Its main purpose is:

- Allow reading data from a device
- Completely block any write operation

So, the original evidence remains unchanged and legally valid.

2. Why Write Blockers are Needed

In digital forensics, even a small change can:

- Alter timestamps
- Modify file system metadata
- Corrupt evidence

Without a write blocker:

- Operating systems may automatically write data (logs, temp files)
- This can destroy original evidence

Therefore: Write blockers ensure integrity and admissibility of evidence in court.

3. How a Write Blocker Works

Step-by-step working:

1. The suspect storage device (HDD/SSD/USB) is connected to the write blocker
2. The write blocker connects to the forensic workstation
3. It allows only read commands to pass through
4. Any write command from system is intercepted and blocked
5. Data is copied safely to another storage device

Key idea: Read allowed → Write denied

4. Types of Write Blockers

1. Hardware Write Blocker

- Physical device placed between disk and computer
- Most reliable and widely used

Example: Tableau write blocker

Features:

- Works independently of OS
- Prevents all write signals at hardware level

2. Software Write Blocker

- Software-based protection
- Configures OS to prevent writes

Example: Disk mounting in read-only mode

Limitations:

- Less reliable than hardware
- Can fail due to OS bugs or misconfiguration

3. Firmware Write Blocker

- Built into device firmware
- Controls read/write at device level

Used in: Specialized forensic tools

5. Key Features of Write Blockers

- Prevents any data modification
- Maintains original evidence integrity
- Supports multiple storage types (SATA, USB, NVMe)
- Logs activity for verification
- Ensures forensic soundness

6. Role in Data Acquisition (Important)

According to the forensic acquisition process:

1. Remove suspect disk
2. Connect it via write blocker
3. Connect target storage for image
4. Create bit-by-bit forensic image
5. Store original safely

Without write blocker → Evidence may be rejected in court

7. Advantages

- Protects original data from changes
- Ensures legal admissibility
- Prevents accidental or automatic writes
- Provides reliable forensic results

8. Limitations

- Hardware blockers can be costly
- May not support all modern interfaces
- Software blockers are less secure
- Requires proper setup and expertise

9. Example (Real-Life Scenario)

Case: Police seize a suspect's hard drive

Without write blocker: Connecting to system may modify files automatically

With write blocker:

- Disk is connected through hardware blocker
- Only data is read
- Forensic image is created
- Original disk remains unchanged

Result: Evidence is preserved and accepted in court

Imaging techniques

Imaging techniques in digital forensics are the methods used to create an exact and verifiable copy of a storage device so that investigators can work on the copy instead of the original evidence. The core goal is to preserve *evidence integrity* while capturing all relevant data, including hidden and deleted portions.

1. Bit-by-Bit (Physical) Imaging

This is the most complete and forensically sound method. It copies every sector of the storage device exactly as it exists.

- Captures active data, deleted files, slack space, and unallocated space
- Does not rely on the file system
- Produces a *true replica* of the disk

Example: Imaging a suspect's hard drive and later recovering deleted photos from unallocated space

Key idea: **Complete duplication of entire disk**

2. Logical Imaging

This method copies only the files and folders visible through the operating system.

- Excludes deleted files and hidden areas
- Faster and requires less storage
- Used when full disk imaging is not necessary

Example: Copying only user documents and emails for quick investigation

Key idea: **Copies only accessible data**

3. Sparse Imaging

This technique captures only selected or relevant parts of the disk instead of the entire storage.

- Skips empty or irrelevant sectors
- Saves time and storage space
- Used in large storage environments

Example: Imaging only sectors containing active files in a 1TB disk

Key idea: **Selective data acquisition**

4. Disk-to-Image File Acquisition

In this method, the entire disk is stored as an image file.

- Common formats: *.dd*, *.E01*
- Easy to store, transfer, and analyze
- Can be mounted like a real disk

Example: Creating an EnCase (*.E01*) image and analyzing it on another system

Key idea: **Disk stored as a file**

5. Disk-to-Disk Imaging (Cloning)

This method directly copies data from one disk to another.

- No intermediate image file

- Requires equal or larger destination disk
- Faster for duplication

Example: Cloning a suspect disk to another hard drive for backup

Key idea: **Direct duplication to another disk**

6. Live Imaging

Performed when the system is still running.

- Captures volatile data like RAM
- Used when shutdown is not possible
- Risk of altering system state

Example: Capturing RAM to detect running malware

Key idea: **Imaging active system**

7. Dead Imaging

Performed when the system is powered off.

- Storage device is removed and imaged
- Uses write blockers
- Most reliable and preferred method

Example: Removing a hard disk and imaging it in a forensic lab

Key idea: **Imaging in controlled environment**

8. Remote Imaging

Data is acquired over a network from a remote system.

- Useful when physical access is not possible
- Requires secure transfer methods

Example: Imaging a remote server involved in a cyberattack

Key idea: **Network-based acquisition**

9. Incremental Imaging

Captures only the changes made since the last image.

- Saves time and storage
- Used in continuous monitoring

Example: Daily imaging of only modified files in a system

Key idea: **Captures recent changes only**

10. Differential Imaging

Captures all changes made since the initial full image.

- Larger than incremental but faster than full imaging
- Used for comparison and tracking

Example: Comparing system state before and after an attack

Key idea: **Tracks changes from baseline**

Evidence integrity

Evidence integrity in digital forensics refers to maintaining the *original state, accuracy, and trustworthiness* of digital evidence from the moment it is collected until it is presented in court. It ensures that the data has **not been altered, tampered with, or corrupted** at any stage of the investigation.

1. Core Meaning

Evidence integrity means:

- The evidence remains *exactly the same* as it was at the time of seizure
- Any change, even a single bit, is detectable
- The evidence can be *trusted and verified* in legal proceedings

In simple terms: **“What was collected = What is analyzed = What is presented.”**

2. Why Evidence Integrity is Important

- Courts only accept evidence that is **reliable and untampered**
- Ensures **fair investigation and justice**

- Prevents *false accusations or manipulated proof*
- Maintains credibility of forensic experts
- Supports *repeatability* (another expert should get same result)

If integrity is compromised → **evidence may be rejected in court**

3. How Evidence Integrity is Maintained

a. Use of Write Blockers

- Prevents any modification to original device
- Allows only *read operations*

b. Hashing (Digital Fingerprinting)

- Generates unique hash values (MD5, SHA)
- If data changes → hash changes
- Used to verify integrity at every stage

c. Chain of Custody

- Records every movement of evidence
- Includes *who handled it, when, where, why*
- Ensures accountability

d. Forensic Imaging

- Create *bit-by-bit copy* of original data
- Analysis is done on copy, not original

e. Secure Storage

- Evidence stored in controlled environment
- Prevents unauthorized access

4. Key Principles of Evidence Integrity

- **No alteration** of original data
- **Traceability** of all actions performed
- **Repeatability** of results
- **Verification** using hashes
- **Documentation** of entire process

5. Example

Case: A suspect's laptop is seized

Step-by-step:

- Investigator connects disk using *write blocker*
- Creates forensic image
- Generates hash of original and copy
- Hash values match → integrity verified
- All actions recorded in chain of custody

Result: Evidence is *trusted and admissible in court*

6. What Happens if Integrity is Violated

- Evidence becomes **unreliable**
- Defense can challenge authenticity
- Court may **reject evidence completely**
- Investigation credibility is damaged

Standard Operating Procedures (SOP) for Acquisition and Preservation of Evidence

SOPs are **formal, step-by-step guidelines** followed by forensic investigators to ensure that digital evidence is collected, handled, and preserved in a *legally acceptable and forensically sound manner*. The main goal is to maintain *evidence integrity, authenticity, and admissibility in court*.

1. Pre-Acquisition Preparation

1. Obtain proper **legal authorization** (search warrant, permission).
2. Define **scope of investigation** (what data is needed).
3. Prepare **forensic tools and equipment** (write blockers, imaging tools).
4. Ensure tools are **validated and calibrated**.
5. Plan acquisition strategy (*live vs dead acquisition*).
6. Maintain readiness of **storage devices for imaging**.

2. Identification of Evidence

1. Identify all possible **sources of digital evidence**:
 - Computers
 - Mobile devices
 - External drives
 - Network devices
2. Note system state (*ON/OFF condition*).
3. Identify volatile vs non-volatile data.
4. Prioritize evidence based on importance.

3. Securing the Scene

1. Restrict **unauthorized access** to devices.
2. Prevent evidence tampering.
3. Disconnect network connections if needed.
4. Photograph and document the setup.
5. Record system condition and environment.

4. Preservation of Evidence

1. Ensure **no modification** of original data.
2. Use **write blockers** before accessing storage.
3. Isolate devices to prevent remote access.
4. Handle devices carefully to avoid damage.
5. Maintain **chain of custody records**.

5. Data Acquisition Process

For Dead Systems (Preferred Method)

1. Power off the system safely.
2. Remove storage devices.
3. Connect via **hardware write blocker**.
4. Create **bit-by-bit forensic images**.
5. Store image on secure storage device.

For Live Systems (If necessary)

6. Capture **volatile data (RAM, processes)** first.
7. Use trusted forensic tools.
8. Minimize system interaction.

6. Verification of Acquired Data

1. Generate **hash values (MD5, SHA)** of original and image.
2. Compare hash values.
3. Matching hashes confirm **data integrity**.
4. Record hash values in documentation.

7. Documentation

1. Record every step taken during acquisition.
2. Include:
 - Date and time
 - Tools used
 - Investigator details
3. Document device details (model, serial number).
4. Maintain logs for repeatability.
5. Prepare initial forensic notes.

8. Chain of Custody Maintenance

1. Record **who handled evidence**.
2. Record **when and where it was transferred**.
3. Ensure secure storage between transfers.
4. Maintain continuous tracking.

9. Secure Storage of Evidence

1. Store original evidence in **tamper-proof containers**.
2. Use controlled access storage rooms.
3. Protect against environmental damage.
4. Keep backup copies of forensic images.

10. Analysis Preparation

1. Perform analysis only on **forensic image (copy)**.
2. Use read-only mode.
3. Preserve original evidence untouched.
4. Ensure repeatability of results.

11. Presentation Readiness

1. Ensure all steps are **well documented**.
2. Maintain a clear evidence trail.
3. Prepare reports for legal presentation.
4. Be ready to justify methods used.

Key Principles of SOP

- **Integrity** → No changes to evidence
- **Authenticity** → Evidence is genuine
- **Repeatability** → Same results if repeated
- **Documentation** → Everything recorded
- **Legal compliance** → Follows law

Example Scenario

Case: Cyber fraud investigation

- Investigator seizes suspect's laptop
- Uses write blocker to connect disk
- Creates forensic image
- Generates hash values
- Stores original safely
- Analyzes only the copy
- Documents every step

Result: Evidence remains **valid and admissible in court**

No.	Differentiator	Cloning	Imaging
-----	----------------	---------	---------

1	Definition	Direct copy of one disk to another disk	Creation of a disk copy stored as an image file
2	Output Form	Physical disk (exact duplicate)	Image file (.dd, .E01, etc.)
3	Storage Requirement	Requires another disk of equal or larger size	Can be stored as a file on any storage media
4	Process Type	Disk-to-disk copy	Disk-to-file copy
5	Flexibility	Less flexible (fixed hardware requirement)	Highly flexible (can store, transfer, duplicate easily)
6	Portability	Not portable (requires physical disk)	Portable (image file can be moved across systems)
7	Analysis	Performed directly on cloned disk	Image can be mounted and analyzed using tools
8	Backup Capability	Limited (single copy unless cloned again)	Multiple copies can be created from image
9	Forensic Usage	Used for quick duplication or backup	Preferred in digital forensics investigations
10	Integrity Verification	Possible but less commonly emphasized	Strongly supported using hash values

Unit 5

1. What is a File System

A file system is a method used to **store, organize, and retrieve data** on a storage device.

- It allows the system or user to access stored data easily
- Without a file system, data would exist as raw, unorganized bytes

Example: When you open a file from a folder, the file system helps locate and retrieve it.

2. Independence from Operating System

- File systems are **largely independent of OS**
- Different OS can support different file systems using drivers

Example:

- Linux can read/write NTFS
- Windows can read Linux EXT (with drivers)

3. File System and Volumes

- File systems are created on **volumes**
- A volume can be:
 - Single partition
 - Multiple partitions (RAID, LVM)

Meaning: A file system does not work directly on disk → it works on logical volumes

4. File System as a Database

- A file system behaves like a **database**
- It stores:
 - Data (actual content)
 - Metadata (information about data)

Key idea:

Data = file content

Metadata = information about file

5. Role of File System

- Provides interface between OS and storage
- Helps OS locate and access files
- Manages storage allocation

6. Metadata in File Systems

Metadata is **data about data**

Includes:

- File location (address)
- File size
- File name
- Directory path

Purpose: Helps system retrieve files correctly

7. Block Addressing

- File system stores data in **blocks (or clusters)** *(page 4)*
- Each file is stored using block addresses

Important:

- Large files use multiple blocks
- Blocks may not be stored continuously

Example: A file may be split across different disk locations

8. File System Components

1. General File System Information

- Structure of file system
- Location of data structures
- Size of data units

2. Content of Files

- Stored in containers like:
 - Clusters
 - Blocks

3. Metadata Structures

Examples:

- FAT → directory entries
- NTFS → MFT (Master File Table)
- Linux → Inodes

4. File Name

- Used to identify files

5. User Quota

- Tracks storage usage

6. File System Journals

- Logs changes for recovery

9. File Names and Paths

- Metadata includes file name and directory path
- Helps users locate files

Types:

- Short names (FAT16 → 8.3 format)
- Long names (modern systems)

Limitation:

- File name length depends on:
 - File system
 - Operating system

10. Clusters, Blocks, and Sectors (Page 7)

- Storage hierarchy:
Sector → Smallest unit
Block/Cluster → Group of sectors
- Windows uses **clusters**
- Linux uses **blocks**

Example: 1 cluster = multiple sectors

11. Cluster Size Concept

- Cluster size can be customized (*page 7*)
- OS may automatically assign it

Example: NTFS typical cluster size = **4096 bytes (4 KB)**

12. Impact of Cluster Size (Page 8)

- Larger clusters → fewer addresses needed
- Improves efficiency for large disks

But: Small files waste space

Example: 1-byte file still occupies **4096 bytes**

This unused space = slack space (important in forensics)

13. Essential vs Non-Essential Data

Essential Data

Required for file operation:

- File location
- File name
- Pointers to metadata

Non-Essential Data

Not required for basic function:

- Access time
- Permissions

14. Types of File Systems (Page 10)

1. NTFS

- Used in Windows
- Advanced features, journaling

2. FAT

- Older file system
- Simple structure

3. Unix File Systems

- EXT, EXT2, EXT3, EXT4
- Used in Linux

4. HFS+ and APFS

- Used in macOS

5. UFS, CDFS

- Specialized systems

15. Key Forensic Importance

- Helps locate deleted files
- Metadata reveals user activity
- Slack space contains hidden data
- Journals show system changes

1. Introduction to FAT File System

FAT (File Allocation Table) is one of the **simplest and oldest file systems** used in operating systems.

- Originally used in **MS-DOS and Windows 9x**
- Later replaced by NTFS in modern Windows
- Still widely used in:
 - USB drives
 - Memory cards
 - Embedded systems

Key reason: **High compatibility across systems**

2. Basic Working Concept

The FAT file system works using:

- **Directory Entries** → store metadata of files File name, size and starting address
- **Clusters** → store actual file data
- **FAT Table** → links clusters together

Each file:

- Has a starting cluster
- Remaining clusters are tracked using FAT

3. FAT Structure Purpose

FAT structure is used to:

- Identify **next cluster of a file**
- Track **free and allocated clusters**
- Work as both:
 - Content tracker
 - Metadata structure

4. Versions of FAT

There are three main versions:

- **FAT12** → small storage devices
- **FAT16** → medium size disks
- **FAT32** → large storage devices

Main difference: Size of entries in FAT table

5. Boot Sector

- Located at **first sector of volume**
- Contains **file system information**
- Part of **reserved area of filesystem**

Important point:

- FAT type (12/16/32) is **not directly stored**
- It is determined using calculations

6. FAT32 Additional Features

FAT32 includes:

- Backup boot sector (usually at sector 6)
- Version number
- FSINFO structure:
 - Next free cluster
 - Total free clusters

7. Layout of FAT File System

FAT has **three main sections**:

1. Reserved Area

- Contains boot sector
- Stores file system metadata

2. FAT Area

- Contains:
 - Primary FAT

- Backup FAT
- Used to track cluster chains

3. Data Area

- Stores:
 - File content
 - Directory content

8. Boot Sector Data Structure (36 Bytes)

Important fields:

- Jump instruction (start execution)
- OEM name
- Bytes per sector (512–4096)
- Sectors per cluster (power of 2)
- Reserved area size
- Number of FATs (usually 2)
- Root directory entries (FAT12/16 only)
- Total sectors (16-bit or 32-bit)
- Media type (fixed/removable)
- FAT size
- Sectors per track
- Number of heads
- Hidden sectors
- Total sectors (32-bit)

9. FAT12/16 Additional Fields

- BIOS drive number
- Boot signature (0x29)
- Volume serial number
- Volume label
- File system type label
- Signature (0xAA55)

10. FAT32 Additional Fields

- Size of FAT (32-bit)

- FAT mirroring control
- Version number
- Root directory cluster location
- FSINFO sector location
- Backup boot sector location
- Volume serial number
- Volume label
- File system type
- Signature (0xAA55)

11. Directory Entry Structure

Each file has a **directory entry (32 bytes)** containing:

- File name (ASCII)
- Attributes
- Creation time/date
- Access date
- Modified time/date
- Starting cluster (high + low bits)
- File size

12. File Attributes (Flags)

Important flags:

- Read-only
- Hidden
- System file
- Volume label
- Directory
- Archive
- Long file name (0x0F)

13. Long File Name (LFN) Structure

LFN entries store:

- Unicode file name
- Sequence number

- Checksum
- Multiple entries for long names

Used to support names longer than 8.3 format.

14. exFAT (Extended FAT)

Modern FAT variant with improvements:

- Supports large volumes
- Uses cluster heap
- Tracks free space efficiently
- Includes:
 - Volume size
 - FAT offset
 - Cluster count
 - Root directory location
 - Status flags

15. Data Representation

- Stored in **little-endian format**
 - Numbers appear reversed
 - Strings appear normal

16. Unused Data Regions

- Certain byte ranges are **unused**
- Often contain:
 - Boot code
 - Error messages

17. Cluster Calculation Concept

Cluster size = Sectors per cluster × Bytes per sector

Example: 16 sectors × 512 bytes = 8192 bytes (8 KB)

18. Key Calculations in FAT

Examples:

- Bytes per sector
- Sectors per cluster
- Total sectors
- FAT size
- Reserved area size

All values are calculated using **hexadecimal to decimal conversion**

19. Redundancy in FAT

- Typically **2 FAT tables**
- Used for:
 - Backup
 - Recovery if one gets corrupted

20. Forensic Importance of FAT

- Easy to analyze due to simple structure
- Helps recover:
 - Deleted files
 - Fragmented data
- FAT entries reveal:
 - File allocation
 - Deletion patterns

Summary

Core Concepts (How it Works)

Directory Entry

Every file/folder gets a 32-byte entry storing: name, size, start cluster, timestamps, attributes.

Cluster

Data is stored in fixed-size chunks called **clusters**. A file can span many clusters.

FAT Structure (the table itself)

A table where each entry = a cluster. It tells you: is this cluster free/used? And **which cluster comes next** in a file. Used for both content tracking AND allocation status.

Three Versions of FAT

FAT12

12-bit entries. Used for floppy disks. Very old.

FAT16

16-bit entries. Early hard drives and USB devices.

FAT32

32-bit entries. Most common today on USB/SD cards.

⚠ There is **no field in the boot sector that says "I am FAT12/16/32"**. The type must be **calculated** from boot sector values (total clusters formula).

Physical Layout of FAT Volume

Reserved Area

Boot sector + FSINFO

Sector 0

FAT Area

Primary + backup FAT

After reserved

Data Area

Files and directory clusters

After FAT area

First: How Hexadecimal → Decimal Works

Hexadecimal is **base 16** number system.

Digits: 0–9, A–F (A=10, B=11 ... F=15)

Formula:

Value = digit × (16^{position})

Why 16?

Because hexadecimal has **16 symbols**, so each position increases by powers of 16.

Also Important: Little Endian (WHY reverse?)

FAT stores data in **little endian format**

Meaning: **Least significant byte comes first**

So:

00 02 → actually means 02 00

Examples

1. Bytes per Sector

Given: 00 02

Step 1: Reverse (because little endian)

→ 02 00

Step 2: Convert

$$0x0200 = (2 \times 16^2) + (0 \times 16^1) + (0 \times 16^0)$$

$$(16^2 = 256)$$

$$\text{So: } 2 \times 256 = \mathbf{512}$$

Answer: 512 bytes

2. Sectors per Cluster

Given: 10

Convert:

$$0x10 = (1 \times 16^1) + (0 \times 16^0)$$

$$= 1 \times 16 = \mathbf{16}$$

Now: Cluster size = $16 \times 512 = 8192$ bytes

Why multiply 512? Because 1 sector = 512 bytes

3. Total Sectors (16-bit)

Given: F8 00

Step 1: Reverse

→ 00 F8

Step 2: Convert

$$0x00F8 = (15 \times 16^1) + (8 \times 16^0)$$

$$= (15 \times 16) + 8$$

$$= 240 + 8 = 248 \text{ sectors}$$

4. FAT Size

Given: E0 3C 00 00

Step 1: Reverse

→ 00 00 3C E0

Step 2: Convert important part:

3C E0

Break:

- $3C = (3 \times 16 + 12) = 60$
- $E0 = (14 \times 16 + 0) = 224$

Now:

$$3C \ E0 = (3 \times 16^3) + (12 \times 16^2) + (14 \times 16^1) + (0 \times 1)$$

Calculation:

$$= (3 \times 4096) + (12 \times 256) + (14 \times 16)$$

$$= 12288 + 3072 + 224$$

$$= 15584 \text{ sectors}$$

5. Reserved Area Size

Given: 40 06

Step 1: Reverse

→ 06 40

Step 2: Convert

$$0x0640 = (6 \times 256) + (4 \times 16)$$

$$= 1536 + 64$$

$$= \mathbf{1600 \text{ sectors}}$$

Ultra Short Memory Trick

Hex → Decimal:

- $\times 1, \times 16, \times 256, \times 4096$

So just remember:

$$\mathbf{1 \rightarrow 16 \rightarrow 256 \rightarrow 4096}$$

1. Introduction to NTFS

NTFS (New Technology File System) is a file system used by Windows OS to store and manage data on storage devices.

- Introduced with **Windows NT 3.1 (1993)**
- Default file system for modern Windows (2000, XP, 7, 8, 10)
- Used in HDDs, SSDs, internal and external storage

Why NTFS was introduced

- FAT had limitations (size, security, reliability)
- NTFS provides:
 - Better **performance**
 - Better **security**
 - Better **reliability**

2. Key Features of NTFS

- Supports **large file sizes** (up to ~16 EB)
- Supports **large volumes** (up to ~256 TB)
- Provides **file permissions** (security)
- Supports **encryption and compression**
- Uses **journaling** to prevent data loss
- Supports **symbolic links and hard links**

Why journaling is important

If system crashes → changes are logged → recovery is possible

- NTFS has a maximum file size of 16 exabytes (EB) and a maximum volume size of 256 terabytes (TB), although the practical limits may be lower depending on the operating system and hardware.

3. What is MFT (Master File Table)

MFT is the **core component** of NTFS

- It is a **database** that stores information about every file and directory
- Created when disk is formatted
- Located at a fixed location
- Expands as files increase

Key Idea: In NTFS: **Everything is stored as a file (including metadata)**

4. Structure of MFT

- MFT is divided into **fixed-size records**
- Each record represents:
 - A file OR
 - A directory

Special Record:

- First record = **Master File Record (MFR)** → stores info about MFT itself

5. MFT Records

Each MFT record contains:

- File name
- Size
- Permissions
- Timestamps
- Attributes

Unique Identifier: Each record has **File Reference Number (FRN)** → used to reference files internally

6. Types of Attributes in MFT

1. Resident Attributes

- Stored inside MFT record
- Used for small data

2. Non-Resident Attributes

- Stored outside MFT
- MFT stores pointer to data

- Attributes can also be standard (defined by NTFS) or non-standard (defined by applications or third-party software).

7. Structure of Attributes

Each attribute has:

- **Header (also the location of the next entry)**
 - Type
 - Length
 - Flags
- **Body:** Actual data

- The actual size of the MFT entry, which includes the header and its attributes, can vary depending on the file's characteristics and the size of its attributes.

8. MFT Record Size

- Depends on **cluster size**
- Default cluster size = **4 KB (4096 bytes)**
- MFT record size is usually multiple of 4 KB

9. Reserved MFT Entries

- First **16 entries reserved** for system files
- Names start with “\$” and the first capital letter.
- Example: \$MFT, \$LogFile

10. Relationships in MFT

- Directory records contain references to other records
- Acts like **linked structure**

Example: Folder → points to files inside it

- MFT records can also contain unused space, which can be used to store additional attributes or data in the future.

What is MFT Entry + Sequence Number

- Every file in NTFS has an **MFT entry (record)**
- Each entry is assigned a **16-bit sequence number**

What is the sequence number?

- It is a counter value
- It **increases every time that MFT entry is reused**

Why it increases

When:

- File is deleted → entry becomes free
- New file uses same entry → sequence number increments

Why Sequence Number is Important

Without sequence number:

- Same MFT entry number could point to **old deleted file**
- This creates confusion

With sequence number:

- System can differentiate between: Old file & New file using same entry

File Reference Address (64-bit)

NTFS does not directly use only MFT entry number
Instead, it creates a **64-bit File Reference Address**

Structure:

- Upper 16 bits → **Sequence Number**
- Lower 48 bits → **MFT Entry Number**

So: **File Reference Address = Sequence Number + MFT Entry Number**

Why Combine Them

Because:

- MFT entry number alone is **not reliable**
- Same entry can be reused many times

Sequence number ensures:

- Each reference is **unique and valid**

How It Helps Detect Corruption

When system accesses a file:

1. It reads file reference address
2. Extracts:
 - Entry number
 - Sequence number
3. It checks:
 - Does the sequence number match current MFT entry?

If YES: → File is valid

If NO: → Entry is reused → old reference → **invalid or corrupted**

Simple Example

Step 1:

- File A stored in MFT entry 100
- Sequence number = 1

Reference = (1, 100)

Step 2:

- File A deleted
- Entry 100 reused

Step 3:

- File B stored in entry 100
- Sequence number becomes 2

Reference = (2, 100)

Now:

If system sees (1,100): → It knows this is **old/deleted file reference**

11. Default MFT Attributes (Very Important)

\$STANDARD_INFORMATION

- Timestamps (create, modify, access)
- Permissions

\$FILE_NAME

- File name
- Parent directory info
- Creation time etc

\$OBJECT_ID

- Unique ID for tracking across different values

\$SECURITY_DESCRIPTOR

- Security permissions like: Owner, access control

\$VOLUME_NAME

- Name of disk/volume and differentiate with others

\$VOLUME_INFORMATION

- Volume details (creation time, serial number, number of sectors etc.)

\$DATA

- Actual file content

12. \$DATA Attribute

- Stores real data
- Can be:
 - Resident (inside MFT)
 - Non-resident (outside MFT)

13. Additional NTFS System Files (Probably non-standard)

1. **\$MFT**: Main table
2. **\$MFTMirr**: Backup of MFT
3. **\$LogFile**: Records file system changes
4. **\$Volume**: Volume info
5. **\$AttrDef**: Attribute definitions
6. **\$Bitmap**: Tracks used/free space
7. **\$Boot**: Boot sector
8. **\$BadClus**: Bad sectors
9. **\$Secure**: Security info

\$FILE_NAME Attribute Data Structure	
Offset	Length
0-7	File reference of parent directory
8-15	File creation time
16-23	File modification time
24-31	MFT modification time
32-39	File access time
40-47	Allocated size of file
48-55	Real size of file
56-59	Flags
60-63	Reparse value
64-64	Length of name
65-65	Namespace
66 +	Name

sarang.rajvansh@nfsu.ac.in School of Cyber Security & Digital Forensics National Forensic Sciences University

\$ATTRIBUTE_LIST Attribute Data Structure		
Offset	Length	Essential
0-3	Attribute type	Yes
4-5	Length of this entry	Yes
6-6	Length of name	Yes
7-7	Offset to name (relative to start of this entry)	Yes
8-15	Starting VCN in attribute	Yes
16-23	File reference where attribute is located	Yes
24-24	Attribute ID	Yes

14. Benefits of MFT

- Fast access (fixed location)
- Reduces fragmentation coz file and directory info is stored in contiguous block of space
- Supports advanced features
- Efficient file management

15. Drawbacks of MFT

- Can become fragmented -> can slow down the access
- Can get corrupted → data loss
- Not compatible with all OS

16. NTFS Forensic Importance

NTFS is very important in investigations because:

- Stores **rich metadata**
- Tracks file activity
- Helps recover deleted files
- \$LogFile contains information about changes made in file system

17. MFT in Forensics

From MFT we can extract:

- File names (even deleted)
- File size
- File location
- Timestamps
- Permissions

Uses:

- Recover deleted files
- Track user activity (browsers, passwords, apps, accounts etc)
- Detect malware
- Can be used in a variety of investigations like cybercrime, financial fraud, child exploitation.

18. NTFS Header Analysis

- First sector (sector 0)
- Contains:
 - Cluster size
 - MFT location

- MFT mirror location

Why important:

- Helps understand disk structure
- Detect tampering

19. NTFS Header Recovery

- Contains backup boot sector
- Used if original is corrupted
- Requires various tools like FTK, encase and dd.

• Anomalies in the NTFS header, such as incorrect values or signatures, can indicate potential tampering or damage to the file system.

• The MFT also contains information on deleted files, which can be useful in forensic investigations.

What actually happens when a file is deleted in NTFS

When a file is deleted the file is **not immediately erased from disk**

- Only:
 - Its **MFT entry is marked as unused**
 - Its clusters are marked as **free**

So: **Data still exists** → **only reference is removed**

What remains in MFT after deletion

Even after deletion, the MFT entry may still contain:

- File name
- File size
- Timestamps (created, modified, accessed)

- File path (sometimes)
- Starting cluster location
- Attribute data

This remains until that MFT entry is **overwritten by a new file**

Why MFT keeps deleted file data

Because NTFS:

- Does not immediately wipe data (for performance reasons)
- Just marks space as **available for reuse**

So: Old data stays temporarily → useful for recovery

Forensic Importance

This is extremely important in investigations:

Investigators can:

- Recover **deleted file names**
- Identify **when file was created/deleted**
- Track **user activity timeline**
- Detect **suspicious file deletion attempts**

How Recovery Works

Forensic tools:

- Scan MFT entries
- Identify entries marked as **deleted**
- Extract remaining metadata

If data clusters are not overwritten: → File can be **fully recovered**

Example

Step 1: User deletes `secret.docx`

Step 2: MFT entry still contains:

- Name: secret.docx

- Size: 2MB
- Created date

Step 3: Investigator checks MFT:

→ Finds deleted file entry

→ Recovers file or at least metadata

7. Limitation

Recovery is possible only if:

- MFT entry is not overwritten
- Data clusters are not reused

Otherwise: → Data is permanently lost

20. \$LogFile Analysis

- Records all file system changes
- Acts as **timeline of activity**

Contains:

- File creation
- Modification
- Deletion

Important:

- Circular log → old data overwritten

- \$LogFile forensic analysis involves extracting and analyzing the contents of the log file to determine the history of activity on the file system.

21. Forensic Value of \$LogFile

- Tracks user actions
- Detects suspicious activity

- Shows file movement and changes

- \$LogFile forensic analysis can reveal valuable information, such as the names and paths of files that were created or modified, the timestamps of these events, and the identities of the users who performed them.

Disadvantage: Large volume of file

- Various techniques can be used to improve the efficiency of \$LogFile forensic analysis, such as filtering the log records based on specific criteria or using timeline analysis to identify patterns of activity over time.

22. MFT Analysis

From MFT we get:

- File names
- Attributes
- Timestamps
- File size
- File location

23. Forensic Uses of MFT Analysis

- Recover deleted files
- Track user behavior
- Identify malicious files
- Reconstruct timeline

24. Challenges in NTFS Forensics

- Complex structure
- Data hiding possible
- Logs may overwrite
- Requires advanced tools for data carving, timeline analysis, keyword searching and metadata analysis.

1. What is \$Bitmap in NTFS

- **\$Bitmap** is a **system file (MFT Record 6)**
- It keeps track of **used and unused clusters** on disk

Core Idea

- Each **bit represents one cluster**
 - 1 → cluster is used
 - 0 → cluster is free

2. Why Bitmap is Needed

NTFS needs a fast way to:

- Allocate space to new files
- Identify free space
- Avoid overwriting existing data

So **\$Bitmap** acts like: “**Disk space manager**”

3. How Bitmap Works

Example:

Bit Position	Value	Meaning
--------------	-------	---------

0	1	Cluster used
1	0	Cluster free
2	1	Cluster used

So system:

- Reads bitmap
- Finds 0 → assigns cluster to file

4. Internal Working Logic

- Bitmap is stored as **sequence of bits**
- Each bit corresponds to: → One cluster number

So: Cluster number = Bit position

5. Forensic Importance of Bitmap

Bitmap helps investigators to:

- Identify **free vs allocated space**
- Detect **deleted files (clusters marked free but data exists)**
- Find **hidden or suspicious data**
- Validate **file allocation consistency**

6. Key Forensic Insight

Important condition:

- If bitmap = 0 (free)
- But cluster contains data → That data is: **Deleted file OR hidden data**

7. Detecting Anti-Forensics

Attackers may:

- Manipulate bitmap
- Mark used clusters as free

Result:

- Data hidden from OS
- But visible in raw analysis

8. Bitmap vs MFT Relationship

- MFT → tells which clusters belong to file
- Bitmap → tells whether cluster is free or used

Cross-checking

If mismatch: → Possible **corruption or tampering**

9. Bitmap Analysis Process

Steps:

1. Extract \$Bitmap
2. Map bits to clusters
3. Compare with: MFT entries
4. Identify anomalies

However, \$Bitmap analysis can be complicated by the fact that clusters may be allocated for alternate data streams or for system files, which are not reflected in the \$Bitmap. Additionally, file compression or encryption can also affect the allocation of clusters on the volume, making it more difficult to determine which clusters are allocated to which files.

NTFS: File Slack & Unallocated Space

1. File Slack

- Occurs when:
 - File does not fully occupy a cluster
- Remaining space = **Slack Space**

Types:

- **RAM Slack**
 - From end of file to end of sector
- **Drive Slack**
 - From end of sector to end of cluster

2. Why Slack is Important

- Can contain:
 - Residual data
 - Old file fragments
 - Useful in investigations for:
 - Recovering hidden information
-

3. Unallocated Space

- Space not assigned to any file
- May contain:

- Deleted files
- Partial data remnants

4. Forensic Importance

- Analysts perform:
 - File carving
 - Keyword searching
- Helps recover:
 - Deleted documents
 - Evidence of past activity

NTFS: Alternate Data Streams (ADS)

1. What is ADS

- NTFS feature allowing:
 - **Multiple data streams in one file**
- Syntax: filename:streamname
- It is used to store metadata, hidden data or thumbnails for better indexing and flexibility and speed.

2. Example

- `file.txt:secret.txt`
- Main file looks normal, but hidden stream contains data

3. Why ADS is Dangerous

- Used for:
 - Hiding malware
 - Concealing data
- Not visible in normal file view

4. Forensic Importance

- Investigators must:
 - Detect hidden streams
 - Use tools like:
 - `streams.exe`
 - forensic suites

Entry	File Name	Description
0	\$MFT	The entry for the MFT itself.
1	\$MFTMirr	Contains a backup of the first entries in the MFT.
2	\$LogFile	Contains the journal that records the metadata transactions.
3	\$Volume	Contains the volume information such as the label, identifier, and version.
4	\$AttrDef	Contains the attribute information, such as the identifier values, name, and sizes.
5	.	Contains the allocation status of each cluster in the file system. Contains the root directory of the file system.
6	\$Bitmap	

Entry	File Name	Description
7	\$Boot	Contains the boot sector and boot code for the file system.
8	\$BadClus	Contains the clusters that have bad sectors.
9	\$Secure	Contains information about the security and access control for the files (Windows 2000 and XP version only).
10	\$Upcase	Contains the uppercase version of every Unicode character.
11	\$Extend	A directory that contains files for optional extensions. Microsoft does not typically place the files in this directory into the reserved MFT entries.

10. What is \$BadClus (Related Concept)

- Tracks **bad/damaged clusters**
- Prevents system from using them

Forensics:

- May hide data intentionally
 - Important to analyze
-

11. What is \$Boot File

- Contains **boot sector information**
- Includes:
 - Cluster size
 - MFT location

Importance:

- Used for **file system recovery**
-

12. What is \$Secure

- Stores **security information**
- Includes:
 - User permissions
 - Access control

Forensics:

- Helps identify **who accessed what**
-

13. What is \$AttrDef

- Defines all **possible attributes**
 - Acts like:
Rulebook for NTFS attributes
-

14. What is \$Volume

- Contains:
 - Volume name
 - Serial number

Used to:

- Identify storage device
-

15. Forensic Use of Bitmap + Other Files

Combined analysis gives:

- File existence
 - Deleted data
 - Hidden data
 - Disk usage pattern
-

16. Timeline Correlation

Bitmap + LogFile + MFT:

- MFT → file metadata
- Bitmap → storage status
- LogFile → changes

Together:

→ Full **timeline reconstruction**

17. Detecting Deleted Files Using Bitmap

Steps:

1. Find cluster marked free
 2. Check raw data
 3. If data exists → deleted file
-

18. Limitations of Bitmap Analysis

- Does not store file names
 - Only shows allocation status
 - Needs MFT for full context
-

19. Challenges in Bitmap Analysis

- Large disk → large bitmap
 - Requires mapping logic
 - Can be manipulated
-

20. Tools Used

- FTK Imager
 - EnCase
 - Autopsy
 - Hex editors
-

21. Combined NTFS Forensic Workflow

1. Analyze Header
 2. Extract MFT
 3. Analyze \$Bitmap
 4. Analyze \$LogFile
 5. Correlate results
-

22. Real Investigation Use

Used in:

- Cybercrime
 - Fraud
 - Data theft
 - Insider attacks
-

23. Final Concept Summary

- **\$Bitmap** = cluster allocation map
- Works at **bit level**
- Critical for detecting:
 - Deleted data
 - Hidden data
 - Disk inconsistencies

Now I'll explain **Fixup Values** exactly based on your provided slides, covering every line + meaning + logic (no missing points).

NTFS Fixup Values (From Your Slides)

1. Basic Concept

- NTFS uses fixup values in **data structures that span multiple sectors**.
- Example:
 - MFT entries
 - Index records

👉 Key idea:

Fixup = **mechanism to ensure multi-sector data is not corrupted**

2. What NTFS Actually Does

Step when writing to disk:

- For each sector in a structure:
 - Take **last 2 bytes**
 - Replace them with a **signature value (Fixup value)**

So:

Original last 2 bytes → saved somewhere

Replaced with → Fixup signature

3. Why This is Done

- When large data structures are written:
 - System crash may interrupt writing
- Result:
 - Some sectors written, some not → corruption

👉 Fixup helps detect this situation

4. Signature Verification Logic

When reading:

- OS checks:
 - Are last 2 bytes of ALL sectors = same signature?

If YES:

- Data is consistent
- Restore original bytes

If NO:

- Data is corrupted

5. Important Rule

- Fixups are used:
 - ONLY in **metadata/data structures**
 - NOT used in:
 - Normal file content sectors
-

6. Internal Structure of Fixup

Each structure using fixup contains:

(A) Signature (Update Sequence Number)

- A **16-bit value**
- Stored in header

(B) Array (Update Sequence Array)**

- Stores:
 - Original last 2 bytes of each sector
-

7. Writing Process (Very Important)

When structure is written:

1. Signature value is **incremented by 1**
 2. Last 2 bytes of each sector:
 - Copied into array
 3. Replace last 2 bytes of each sector:
 - With signature
-

8. Reading Process

When OS reads:

1. Check last 2 bytes of each sector
2. Compare with signature

If match:

- Replace them with original values from array

If mismatch:

- Data is corrupted
-

9. Corruption Detection Case

If:

- Only one sector was written (partial write)

Then:

- That sector will NOT match signature

👉 OS concludes:

- **Data structure is corrupted**
-

10. What Fixups Detect

- Damaged sectors
 - Partial writes
 - Corrupt data structures
-

11. MFT-Specific Point (Very Important)

- Every **MFT entry uses fixup values**
- Meaning:
 - On disk:
 - Last 2 bytes of each sector in MFT record = replaced by fixup

👉 So:

- Raw MFT data is always **modified with fixup before storing**

12. Simple Visual Flow

WRITE:

Sector end bytes → saved in array

Replace with → signature

READ:

Check signature match

→ Yes → restore original bytes

→ No → corruption

13. Easy Example

Assume:

- Signature = AA BB

Original sector endings:

Sector1 → 11 22

Sector2 → 33 44

After writing:

Sector1 → AA BB

Sector2 → AA BB

Array → [11 22, 33 44]

During read:

- If both sectors end with AA BB → OK
- Restore:

Sector1 → 11 22

Sector2 → 33 44

If mismatch:

- Example:

Sector2 → 99 88

👉 Means corruption

14. Why This is Critical in Forensics

Fixup values help investigators:

Detect:

- Disk corruption
- Interrupted writes
- Tampering

Validate:

- MFT integrity
 - Metadata reliability
-

15. Direct Exam Points

- Fixup = integrity check for multi-sector structures
- Replaces last 2 bytes of each sector
- Uses:
 - Signature (USN)
 - Array (USA)
- Used in:
 - MFT entries
- Detects:
 - Partial writes
 - Corruption

Here is a **complete, structured explanation of the entire PDF (without missing any point)** in a clean, easy-to-revise format.

Windows Registry & Event Logs

1. Windows Registry – Basic Understanding

Definition

- Windows Registry = **central hierarchical database**
- Stores:
 - System configuration
 - User settings
 - Application data
 - Hardware details

Purpose

- Replaced old **.ini configuration files**
- Tracks user activity to:
 - Optimize system behavior

- Maintain personalization

2. Registry Key Terms (Core Concepts)

Registry Components

- **Registry** → Collection of system + user info files
- **Registry Editor** → Tool to view/edit registry
 - Regedit
 - Regedt32 (from Windows 2000)

Structural Terms

- **HKEY** → Top-level categories (like root folders)
 - Windows 9x → 6 types
 - Windows 2000+ → 5 types
 - “H” = handle (programming reference)
- **Key** → Folder inside HKEY
- **Subkey** → Folder inside another key
- **Branch** → Key + all its subkeys
- **Value** → Data stored in a key (like file content)
- **Default Value** → Every key has one (may be empty)

3. Registry Hives

What are Hives?

- Logical group of registry data
- Stored as files on disk

Important Points

- Located mainly in:
 - `Windows\System32\Config`
 - User profiles

Key Hive Files

- **NTUSER.DAT** → User-specific data
- **USRCLASS.DAT** → Shell & UI preferences
- **SAM, SYSTEM, SOFTWARE, SECURITY**

Internal Structure

- Begins with header: **regf**
- Contains blocks called: **hbin**
- Deleted entries can be recovered (forensics)

4. Important Registry Files (Forensics Focus)

System-wide files (in config folder)

- **SYSTEM** → OS configuration
- **SOFTWARE** → Installed programs
- **SAM** → User accounts & passwords
- **SECURITY** → Security policies

User-specific files

- **NTUSER.DAT**
 - MRU (Most Recently Used)
 - URLs, commands, apps used
- **USRCLASS.DAT**
 - ShellBags (folder view preferences)

5. Explanation of Major Registry Files

SOFTWARE Hive

- Stores:
 - Installed apps & miscrosoft apps
 - OS data
- 64-bit systems:
 - **WOW6432Node** → 32-bit app compatibility

SYSTEM Hive

- Contains:
 - Control Sets:
 - CurrentControlSet

- LastKnownGood
- Important for:
 - Boot configuration
- Forensics: Always check **multiple control sets**

SAM (Security Accounts Manager)

- Stores:
 - Local user accounts
 - Encrypted passwords
 - User hive locations
- Domain users → stored in Active Directory

SECURITY Hive

- Stores:
 - Security policies
 - Domain security database
 - Used by kernel to enforce security rules
-

NTUSER.DAT

- Per-user file
- Contains:
 - MRU files
 - Typed URLs
 - Commands
 - Installed apps
 - File shares

USERCLASS.DAT

- Introduced in Windows 7
- Stores:
 - ShellBags
 - Folder size
 - View type
 - Sorting

6. Registry Hive Structure (Technical)

Physical Structure

- Made of **4KB bins**
- First bytes: **regf**

Important Hives

- HKCR → Classes
- HKCU → Current user
- HKLM → Local machine
- HKU → All users
- HKCC → Hardware profile

7. Explanation of Each HKEY

HKCR (HKEY_CLASSES_ROOT)

- File associations (.exe etc.)
- COM objects
- Automation scripts

HKCU (HKEY_CURRENT_USER)

- Current logged-in user data
- Includes:
 - Desktop settings
 - Network settings
- Acts as pointer to HKU
- Changes each time the user logs in to the system.

HKLM (HKEY_LOCAL_MACHINE)

- System-wide settings
- Hardware + OS configuration
- Most important forensic hive

HKU (HKEY_USERS)

- Contains all user profiles
- Includes:
 - **.DEFAULT: where no users are logged in**
 - SID-based keys: current logged in user

- Classes key

HKCC (HKEY_CURRENT_CONFIG)

- Hardware profile at runtime
- Pointer to: HKLM\System\CurrentControlSet\Hardware Profiles
- Contains:
 - Display settings
 - BIOS info

8. Registry Behavior (Important Concepts)

Volatile Data

- Exists only during runtime
- Not saved on disk

Example: HKCU is rebuilt from NTUSER.DAT

Example Case

- **ProgramCount** value:
 - Shows running apps
 - Exists only during runtime
 - Not found after shutdown

9. Auto Startup Locations

Purpose

- Programs run automatically on boot

Forensics Insight

- Malware uses registry to persist

Key Locations

- HKLM...\Run
- HKCU...\Run
- RunOnce, RunServices, etc.

Registry stores: Every startup program

10. Applications & Registry

- Not all apps use registry
- Example:
 - Java-based apps
 - P2P tools
- Use config files instead

11. Registry Structure (Logical View)

Components

- Hives
- Keys
- Subkeys
- Values
- Data

Value Types

- Binary
- String
- Hex
- BLOB

12. Registry Key Internal Structure (Forensics)

Key Cell (nk)

Contains:

- Size (DWORD)
- Node ID = "nk"
- Node type:
 - 0x2C → Root key

- 0x20 → Normal key
- LastWrite time (FILETIME)

LastWrite Time

- 64-bit timestamp
- Counts from: Jan 1, 1601
- Used in timeline analysis

Key Cell Fields

- Parent key offset
- Number of subkeys
- Number of values
- Security descriptor
- Key name length

13. Registry Value Structure

Value Cell (vk)

Contains:

- Size
- Node ID = "vk"
- Value name length
- Data length
- Offset to data
- Value type

14. Registry Value Types (Complete)

Type	Name	Description
0	REG_NONE	No type

1	REG_SZ	String
2	REG_EXPAND_SZ	String with variables
3	REG_BINARY	Raw data
4	REG_DWORD	32-bit
5	REG_DWORD_BIG_ENDIAN	32-bit BE
6	REG_LINK	Symbolic link
7	REG_MULTI_SZ	Multiple strings
8	REG_RESOURCE_LIST	Hardware map
9	REG_FULL_RESOURCE_DESCRIPTOR	Hardware description
10	REG_RESOURCE_REQUIREMENTS_LIST	Driver info
11	REG_QWORD	64-bit

15. Important Registry Locations (For Persistence Analysis)

System-level

- HKLM\System\CurrentControlSet\Services
- HKLM\Software...\Run
- HKLM...\ShellServiceObjects

User-level

- HKCU\Software...\Run
- HKCU...\RunOnce
- HKCU...\Policies\Explorer\Run

16. Importance of Registry in Forensics

- One of the **richest evidence sources**
- Helps in:
 - Timeline reconstruction
 - User activity tracking
 - Malware detection
- Used to **validate investigation findings**

17. Event Logs (Next Section)

Types of Logs

- System
- Security
- Application
- Setup
- Forwarded Events

18. Event Log Details

Application Logs

- Generated by apps

Security Logs

- Login attempts
- Privilege usage
- Controlled by audit policy

System Logs

- OS-level events

Setup Logs

- Installation / upgrade events

Forwarded Logs

- Logs from remote systems

19. Log Storage

- Stored in:
 - `System32\winevt\Logs`
- Format:
 - XML

Registry Link

- `HKLM\SYSTEM\CurrentControlSet\Services\EventLog`

20. Event Log Levels

- Information
- Warning
- Error
- Critical
- Success/Failure Audit

21. Event Log Fields (Artifacts)

Core Fields

- Level → severity
- Channel → category
- Computer → system name
- Event ID → unique identifier

Advanced Fields

- Keywords → grouping
- Opcode → action performed
- Provider GUID → unique source ID

Additional Fields

- Provider Name
- Security User ID (SID)
- Task category
- Record order
- File offset
- Record ID
- XML view
- Record length

22. Event Log Structure (Binary)

Header Fields

- Length
- Signature ("LfLe")
- Record number
- Time generated
- Time written
- Event ID
- Event type
- Category

Data Section

- Strings
- Event description
- Variable-length fields